

第 23 章: Module Coding Basics (模块编码基础)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 第 22 章讲了模块的"概念模型"(模块如何被查找、加载), 本章进入"实操"——教你亲手创建模块文件、使用 import 与 from 导入、查看模块属性字典 dict、用 reload 热重载模块。学完本章, 你将彻底理解 Python 程序如何由多个文件协作而成。这也是下一章"包导入 (package imports)"的铺垫。

23.1 开篇引言

英文原文

Now that we've studied the larger ideas behind modules, let's turn to some examples of modules in action. Although some of the early topics in this chapter will be review for linear readers who have already applied them in previous chapters' demos, even simple modules can quickly lead us to further details that we haven't yet encountered in full, such as nesting, reloads, scopes, and more, which we'll pick up here. In general, Python modules are easy to create; they're just files of Python program code created with a text editor

, and require no special syntax. Because Python does all the work of finding and loading modules, they are also easy to use; simply import a module or its names, and use the objects they reference. Let's explore both sides of this fence.

中文翻译

在研究了模块背后的大概念之后，现在让我们转向一些模块实际运用的例子。虽然本章前半部分的内容，对按顺序阅读、并已在前面章节演示中实际用过的读者来说算是复习，但即便是最简单的模块，也能很快带我们触及一些尚未完整遇到的深层细节——比如嵌套（nesting）、重载（reloads）、作用域（scopes）等等，这些我们都会在这里讲到。总的来说，Python 模块很容易创建：它们只是用文本编辑器写成的 Python 程序代码文件，不需要任何特殊语法。因为查找和加载模块的工作全由 Python 完成，模块也很容易使用：只需 import 一个模块或它的名字，然后使用它们所引用的对象即可。让我们来看看这道“篱笆”的两边。

深度理解

- 核心概念：本章是模块的“从文件到对象”之旅。一句话总结：任何 .py 文件天然就是一个模块——创建模块零语法成本，使用模块只需要 import。
- 底层视角：“Python does all the work of finding and loading”指的是第 22 章讲过的导入三步流程：查找（find）→ 编译（compile）→ 运行（run）。你只管写文件，解释器负责把文件名变成内存里的模块对象。

·设计思想：Python 把"代码组织"问题简化成了"文件系统"问题。文件本身就是命名空间，目录本身就是包——不需要 XML 配置、不需要构建清单（build manifest），这与 Java/C++ 的构建体系形成鲜明对比。

·实际问题：这解释了为什么 Python 项目结构如此直观：`utils.py`、`config.py`、`models.py`.....任何文件拿来就能 `import`。

·初学者误区：很多人以为"模块"是某种需要特殊声明的东西。实际上模块没有声明语法——你写过的每一个 `.py` 文件（包括练习代码）都已经是模块了。

23.2 创建模块 (Creating Modules)

英文原文

To define a module, simply use your text editor to type Python code into a text file, and save it with a `.py` extension; any such file is automatically considered a Python module. As we've seen, all the names assigned at the top level of the module become its attributes (names associated with the module object) and are exported for clients to use—they morph from variable to module object attribute automatically. For instance, if you type the code in Example 23-1 into a file called `module1.py` and import it, you create a module object with one attribute—the name `printer`, which happens to be a reference to a function object. Example

23-1. module1.py python def printer(x): # Module attribute print(x) Again, this code may seem simplistic if you read the content of this book that precedes this chapter, but our goal here is to strip out the extraneous bits so we can study modules in isolation.

中文翻译

要定义一个模块，只需用文本编辑器把 Python 代码敲进一个文本文件，并以 .py 扩展名保存；任何这样的文件都会被自动视为一个 Python 模块。如我们所见，在模块顶层赋值的所有名字都会成为它的属性（attribute，与模块对象关联的名字），并导出供使用者使用——它们会自动从“变量”变身成“模块对象的属性”。例如，如果你把示例 23-1 的代码输入到一个名为 module1.py 的文件中并导入它，你就创建了一个带有一个属性的模块对象——属性名是 printer，它恰好是对一个函数对象的引用。示例 23-1. module1.py python def printer(x): # 模块属性 print(x) 再说一遍，如果你读过本书本章之前的章节，这段代码可能显得过于简单，但我们这里的目标是把无关紧要的部分剥掉，以便单独研究模块。

深度理解

- 核心概念：文件 = 模块，顶层赋值 = 模块属性。“变量变身属性”是本章最关键的一句话：def printer(x) 在模块顶层执行后，名字 printer 就自动成为模块对象的属性。
- 底层实现：导入时，Python 创建一个空的 module 对象，然后从上到下逐条执行文件里的语句。每条顶层赋值语

句（包括 `def`、`import`、`x = 1`）都等价于往模块对象的内部字典（dict）里塞进一个“名字 → 对象”的键值对。

·设计原因：Python 坚持“无需声明”哲学——不要求 `module` 关键字、不要求导出清单。你写的名字默认都是“公开的”，这就是模块的默认导出机制。

·实际问题：把可复用的函数、常量放进 `.py` 文件，别的文件 `import` 即可用，这是 Python 最简单的代码复用方式。

·初学者误区：误以为函数/类需要特殊写法才能被外部使用。其实只要在模块顶层定义，`printer` 天然就是属性，无需 `export` 之类的声明。

代码分析 (Example 23-1)

```
def printer(x):    #  
    print(x)
```

·解释器如何处理：当 `import module1` 执行时，解释器把 `module1.py` 编译成字节码并运行，运行到 `def` 语句时创建一个函数对象，然后把名字 `printer` 绑定到该函数对象上，存入模块的命名空间字典——于是“变量”瞬间变成“模块对象的一个属性”。

·设计思想：`def` 在这里既是“定义函数”又是“创建模块属性”，两种身份是同一件事。这体现了 Python 的统一性：赋值即绑定，顶层绑定即导出。

23.3 模块文件名 (Module Filenames)

英文原文

Before we go on, let's get more formal about module filenames. You can call modules just about anything you like, but module filenames should end in a `.py` suffix if you plan to import them as modules. The `.py` is technically optional for top-level files that will be run but not imported (i.e., for scripts), but adding it in all cases makes your files' types more obvious, may enable Python-specific features in some text editors and file explorers, and allows you to import any of your files in the future (recall that this is one way to run a file). Because module names become variable names inside a Python program (without the `.py`), they should also follow the normal variable name rules outlined in Chapter 11. For instance, you can create a module file named `if.py`, but you cannot import it because `if` is a reserved word—when you try to run `import if`, you'll get a syntax error. In fact, both the names of module files and the names of directories used in package imports (discussed in the next chapter) must conform to the rules for variable names presented in Chapter 11; they may, for example, contain only letters, digits, and underscores. Package directories also cannot contain platform-specific syntax such as spaces in their names. When a module is imported, Python maps the

module name to an external filename by adding a directory path from the module search path of the last chapter to the front, and a .py or other extension at the end. For instance, a module named M ultimately maps to an external file `directory/M.extension` that contains the module's code.

中文翻译

继续之前，让我们把模块文件名的规则讲得更正式一些。模块可以随便起名，但如果你想把它当作模块来 `import`，文件名最好以 `.py` 后缀结尾。对只运行、不被导入的顶层文件（也就是脚本 `scripts`）来说，`.py` 在技术上是可以省略的；但一律加上它，能让文件类型更明显，可能让某些文本编辑器和文件资源管理器启用 Python 专属特性，也让你将来能导入自己的任何文件（回想一下，导入也是运行文件的一种方式）。因为模块名会变成 Python 程序内部的变量名（去掉 `.py` 之后），所以它们还必须遵循第 11 章列出的普通变量命名规则。比如，你可以创建一个名为 `if.py` 的模块文件，却无法导入它，因为 `if` 是保留字——当你运行 `import if` 时会得到语法错误。事实上，模块文件的名称，以及包导入（package imports，下一章讨论）中用到的目录名字，都必须符合第 11 章的变量命名规则；例如，它们只能包含字母、数字和下划线。包目录的名字里也不能包含空格这类平台相关的字符。当模块被导入时，Python 会把模块名映射为外部文件名：在开头加上上一章讲过的模块搜索路径（module search path）中的某个目录路径，在结尾加上 `.py` 或其他扩展名。例如，名为 `M` 的模块最终映射到外部文件 `directory/M.extension`，其中包含该模块的代码。

深度理解

·核心概念：模块名 = 去掉 .py 的程序内变量名。所以 if.py 能存在但永远无法 import——if 是保留字，import if 在语法层面就非法。

·底层实现——import 的查找路径 (sys.path)：第 22 章说的"模块搜索路径"在代码里就是 sys.path——一个目录字符串列表。import M 时，Python 按下述顺序执行：

1. 先查 sys.modules ("已导入模块表"字典)。命中则直接返回已有模块对象，不重新加载——这就是本章后面"导入只发生一次"的底层原因；
2. 未命中则按 sys.path 从左到右依次尝试在目录名/M.py (或 .pyc、包目录等) 找到文件；
3. 找到后编译、执行，把模块对象放入 sys.modules 并返回。

sys.path 通常包含：脚本所在目录 (或当前目录)、PYTHONPATH 环境变量指定的目录、标准库目录、site-packages (第三方库安装目录)。你随时可以在 REPL 里敲 `import sys; sys.path` 亲眼查看。

·设计原因：把"模块名 → 文件路径"的映射规则固定下来，Python 才能从纯文本名字出发定位文件；而强制变量名规则是为了让 import if 这类歧义在语法层就死掉。

·实际问题：命名模块时避免与标准库同名 (如别写 string.py、math.py)，否则你的文件会遮蔽标准库模块；文件

名用小写加下划线是社区惯例。

·初学者误区：以为“扩展名不重要”或“什么文件都能导入”；其实：脚本可以没 .py，但要 import 就最好有；带空格、连字符（如 my-mod.py）、以数字开头的文件名都无法作为模块名导入。

23.4 其他类型的模块 (Other Kinds of Modules)

英文原文

As mentioned in the preceding chapter, it is also possible to create a Python module by writing code in an external language such as C, C++, and others (e.g., Java, in the Jython implementation of the language). Such modules are called extension modules, and they are generally used to wrap up external libraries for use in Python scripts or optimize parts of a program. When imported by Python code, extension modules look and feel the same as modules coded as Python source code files—they are accessed with import statements, and they provide functions and objects as module attributes. They're also beyond the scope of this book; see Python's standard manuals for more details.

中文翻译

正如前一章提到的，也可以用外部语言编写代码来创建 Python 模块，比如 C、C++ 及其他语言（在 Jython 这个语言实现中还可以用 Java）。这类模块被称为扩展模块（extension modules），通常用来包装外部库供 Python 脚本使用，或优化程序的某些部分。当被 Python 代码导入时，扩展模块与用 Python 源码文件写成的模块看起来、用起来完全一样——用 import 语句访问，以函数和对象的形式作为模块属性提供功能。不过它们超出了本书的范围；更多细节请参阅 Python 的标准手册。

深度理解

- 核心概念：模块不一定用 Python 写。用 C/C++ 写的"模块"叫扩展模块，导入方式与普通模块完全一致。
- 底层实现：在 CPython 里，扩展模块通常是编译好的共享库（Windows 上是 .pyd，Linux/macOS 上是 .so），导出 C 层的模块初始化函数。import 时解释器加载该库、调用初始化函数，得到一个标准 module 对象，之后的一切用法与 .py 模块无异。
- 设计原因：一是复用已有的 C/C++ 库（如 NumPy 底层就是 C 实现的数组运算），二是性能优化——把热点循环下沉到 C。
- 实际问题：你用 import numpy、import zlib 时，拿到的就是扩展模块；对使用者来说它跟普通模块没有区别。
- 初学者误区：以为"Python 模块只能是 .py 文件"。扩展模块的存在说明：模块是一种接口约定，不是文件格式。

23.5 使用模块 (Using Modules)

英文原文

On the other side of the fence, clients can use the simple module file we just wrote by running an `import` or `from` statement. Both statements were introduced in Chapter 3, and have been used in earlier examples. They both find, compile, and run a module file's code if it hasn't yet been loaded, per the process covered in the prior chapter. The chief difference is that `import` fetches the module as a whole, so you must qualify to fetch its names, whereas `from` fetches (really, copies) specific names out of the module. Let's see what this means in terms of code. All of the following examples wind up calling the `printer` function defined in module file `module1.py` of Example 23-1 but in different ways.

中文翻译

在篱笆的另一边，使用者可以通过运行 `import` 或 `from` 语句来使用我们刚写好的那个简单模块文件。这两个语句都在第 3 章介绍过，前面的示例里也用过。它们都会按上一章讲过的流程，在模块尚未被加载时查找、编译并运行模块文件中的代码。主要的区别在于：`import` 把模块整体取过来，你必须通过限定名（qualify）来取其中的名字；而 `from` 是把模块中特定的名字取过来（准确地说是“复制”过来）。

让我们看看这在代码上意味着什么。下面的所有示例最终都会调用示例 23-1 中 `module1.py` 文件里定义的 `printer` 函数，只是方式不同。

深度理解

- 核心概念：两种导入哲学——`import` 给你"整个模块"，属性要加前缀；`from` 给你"选中的名字"，直接用、无前缀。
- 底层实现：两个语句共享同一条底层导入管线：查 `sys.modules` → 沿 `sys.path` 查找 → 编译 → 执行 → 缓存。`from` 只是在整条管线跑完之后，多了一步"复制名字"。
- 设计原因：`import` 保证命名空间隔离（`module1.printer` 不会与你的 `printer` 冲突）；`from` 换来了打字的便利，代价是丢失"出处"。
- 实际问题：经验法则——库、正式代码多用 `import`；REPL 里图省事多用 `from`。
- 初学者误区：误以为 `from` 只加载用到的部分、比 `import` 快。实际两者加载的是同一个完整模块，性能没有差别。

23.5.1 `import` 语句 (The `import` Statement)

英文原文

In the first example that follows, the name `module1` serves two different purposes—it identifies an external file to be loaded, and it becomes a variable in the script, which references the module object after the im

port. In a REPL: `python >>> import module1 # Get module as a whole (one or more)` `>>> module1.printer('Hello world!')` # Qualify to get names Hello world! The import statement simply lists one or more names of modules to load, separated by commas. Because it gives a name that refers to the whole module object, we must go through the module name to fetch its attributes (e.g., `module1.printer`).

中文翻译

在下面的第一个示例中，名字 `module1` 扮演两个不同角色——它标识一个要被加载的外部文件，同时它又变成脚本里的一个变量，在 `import` 之后引用那个模块对象。在 REPL 中：`python >>> import module1 # 整体取得模块（可以是一个或多个）` `>>> module1.printer('Hello world!')` # 限定名来取名字 Hello world! `import` 语句只是列出要加载的一个或多个模块名，用逗号分隔。因为它给出的是一个引用整个模块对象的名字，所以我们必须通过模块名来取它的属性（例如 `module1.printer`）。

深度理解

·核心概念：`import module1` 是“一次赋值”：把模块对象绑定给变量 `module1`。该变量名既指文件（加载源），又指对象（加载结果）。

·底层实现：`import` 语句执行的字节码大致等价于：查找并加载模块 → 执行模块代码 → 把模块对象赋给变量。限定访问 `module1.printer` 则是“先取变量 `module1` 的对

象，再取该对象的 `printer` 属性"——它走的是对象属性查找，与作用域（LEGB）无关。

·设计原因：强制限定访问，让每个名字都有"出处"，避免几十个模块导入后名字互相踩踏。

·实际问题：`import` 一个模块时其实还可以逗号分隔多个：`import os, sys, math`。

·初学者误区：`import module1` 之后直接写 `printer('Hello world!')` 会抛 `NameError`——模块名不经过限定就不会暴露属性，这是特性不是 bug。

23.5.2 from 语句 (The from Statement)

英文原文

By contrast, because `from` copies specific names from one file over to another scope, it allows us to use the copied names directly in the script without going through the module (e.g., `printer`):

```
python >>> from module1 import printer # Copy out a variable (one or more)
>>> printer('Hello world!') # No need to qualify name (and can't!)
Hello world!
```

This form of `from` allows us to list one or more names to be copied out, separated by commas. Here, it has the same effect as the prior example, but because the imported name is copied into the scope where the `from` statement appears, using that name in the script requires less typing—we can use it directly instead of naming the enclosing module. In fact, we must; `from` doesn't assign th

e name of the module itself, so module1 is undefined. As you'll see in more detail later, the from statement is really just a minor extension to the import statement—it imports the module file as usual (running the full three-step procedure of the preceding chapter), but adds an extra step that copies one or more names (not objects) out of the file. The entire file is loaded, but you're given names for more direct access to its parts.

中文翻译

相比之下，因为 from 会把特定的名字从一个文件复制到另一个作用域，它让我们能在脚本里直接使用复制来的名字，而不必经过模块（例如 printer）：
`python >>> from module1 import printer # 复制出一个变量（可以是一个或多个）>>> printer('Hello world!') # 无需限定名（也不能限定！）Hello world!`
这种形式的 from 允许我们列出要复制的一个或多个名字，用逗号分隔。这里它的效果和前面的示例相同，但因为被导入的名字复制进了 from 语句所在的作用域，在脚本里用这个名字就少敲很多字——我们可以直接使用它，而不必写出所在模块的名字。事实上是必须如此：from 不会给模块本身的名字赋值，所以 module 1 是未定义的。稍后你会更详细地看到，from 语句其实只是 import 语句的小小扩展——它照常导入模块文件（完整运行上一章的三步流程），但额外加了一步：把文件中的一个或多个名字（不是对象）复制出来。整个文件都被加载了，只是你拿到的是能更直接访问其内部成员的名字。

深度理解

·核心概念: `from = import + 复制名字`。注意是复制名字 (引用), 不是复制对象——这一点是本章最重要的语义之一。

·底层实现: `from module import printer` 在字节码层面等价于三步: 导入模块 → 从模块对象中取出 `printer` 属性 → 在当前作用域绑定同名变量。模块名 `module1` 从未绑定, 所以它是未定义的 ("`can't!`"——你根本没法用 `module1.printer`, 因为 `module1` 这个名字不存在)。

·设计原因: 少打字 + 直接可用, 是 REPL 交互和脚本书写的巨大便利。

·实际问题: `from` 后面同样可以列多个名字: `from math import sqrt, pi, sin`。

·初学者误区: 两大误解: ①以为 `from` 之后还能用模块名 (不能, 模块名未定义); ②以为 `from` 只加载部分代码 (错误——整个文件照常完整加载、完整执行)。

23.5.3 `from` 语句 (The `from` Statement)

英文原文

Finally, the next example uses a special form of `from`: when we use a `\` instead of specific names, we get copies of all names assigned at the top level of the referenced module. Here again, we can then use the copied name `printer` in our script without going through the module name: `python >>> from module1 import`


```
# Copy out all variables >>> printer('Hello world!') #
```

Use names unqualified Hello world! Technically, both `import` and `from` statements invoke the same import operation; the `from` form simply adds an extra step that copies all the names in the module into the importing scope. It essentially merges one module's namespace into another, which again means less typing for us, albeit at the expense of name segregation. Note that only a single `*` works in this context; you can't use arbitrary pattern matching to select a subset of names (you could in principle by looping through a module's `__dict__` discussed ahead, but it's substantially more work). Also, note the `from` may not really get "all" names in the module if that module uses special tricks like `__all__` naming or an `__all__` list to hide some of its names from this statement, but we'll defer to Chapter 25 for more on such tools. And that's it—apart from the search-path configurations of the prior chapter, modules really are simple to use. To give you a better understanding of what really happens when you define and use modules, though, let's move on to look at some of their properties in more detail. NOTE — OK, there is a special case: The `from` form of the `from` statement form described here can be used only at the top level of a module file, not within a function (and generates a syntax error there), because this would make it impossible for Python to detect local variables before the function runs. It's rare to see either `import` or `from`

m inside a function anyhow, and best practice recommends listing all your imports at the top of a module file; it's not required, but makes them easier to spot, and avoids a speed penalty for re-importing modules on every call to a function. The `from \` has other issues enumerated ahead, and may be best limited to one per file, or interactive REPLs.

中文翻译

最后，下一个示例用了 `from` 的一种特殊形式：当我们用 `\` 代替具体名字时，得到的是被引用模块顶层赋值的所有名字的副本。同样，我们可以在脚本中不经过模块名而直接使用复制来的 `printer`：

```
python >>> from module1 import  
# 复制出所有变量>>> printer('Hello world!') # 不局限  
地使用名字Hello world! 从技术上来说，import 和 from  
语句调用的都是同一个导入操作；from \ 只是额外加了一步：把模块里的所有名字复制进当前作用域。它本质上把一个模块的命名空间合并进了另一个，这同样意味着我们少打字，代价是失去了名字的“隔离性”（name segregation）。注意在这个语境下只能使用单个 \；你不能用任意的模式匹配来选择名字的子集（原则上可以通过遍历模块的 dict 来做——后面会讲到——但那工作量大得多）。另外请注意，如果模块使用了 \X 命名或 all 列表这类特殊技巧把一些名字藏起来，from \ 实际拿到的可能并不是模块里“所有”名字；关于这类工具我们留到第 25 章再讲。就这样——除了上一章的搜索路径配置之外，模块真的很好用。不过，为了让你更好地理解定义和使用模块时究竟发生了什么，让我们继续深入看看模块的一些属性细节。注意（NOTE）——
```

—好吧，有一个特殊情况：这里描述的 `from` 的 `\` 形式只能出现在模块文件的顶层，不能出现在函数内部（在函数里会引发语法错误），因为那会让 Python 无法在函数运行前检测出局部变量。其实把 `import` 或 `from` 放进函数里本来就很少见，最佳实践是建议把所有导入都列在模块文件顶部；这不是硬性要求，但让它们更容易被发现，还能避免“每次调用函数都重新导入模块”带来的速度损失。`from \` 还有前面列举的其他问题，最好限制为每文件至多一次，或者只在交互式 REPL 中使用。

深度理解

·核心概念：`from module import` 是“命名空间合并器”——把对方顶层所有名字一股脑复制进来。它是三种导入形式中最“暴力”的一种。

·底层实现：解释器遍历模块 `dict` 中所有不以开头（且不在 `all` 排除范围）的名字，逐个复制到当前作用域。它受数据隐藏约定（`X` 前缀、`all` 列表）约束，所以“所有”要打引号。

·设计原因：为什么允许？历史上是为了从数学/数值模块里“倒出一袋子工具”方便交互实验；为什么限制为单个？因为模糊匹配会破坏可读性和确定性。

·实际问题：`from math import` 在 REPL 里方便，但在正式代码里你会失去“这个 `sqrt` 是哪来的”的信息；多文件都用 `from` 时，名字来源彻底成谜。

·初学者误区：①以为 `from X import` 比显式导入快——不会，加载成本完全一样；②以为可以在函数里用它——

会直接 `SyntaxError`；③以为它一定复制全部名字——`X` 开头和 `all` 会屏蔽它。

·为什么函数里禁止 `from`：函数执行前，Python 必须静态确定局部变量集合（这决定名字是局部还是全局查找）。`from` 会在运行时才引入名字，破坏了这个静态分析，所以语法上直接禁止——这正是解释器“先静态后动态”的体现。

23.6 导入只发生一次 (Imports Happen Only Once)

英文原文

One of the most common questions people seem to ask when they start using modules is, "Why won't my imports keep working?" They often report that the first import works fine, but later imports during an interactive session (or program run) seem to have no effect. In fact, this is by design—modules are loaded and run on the first import or `from`, and only the first. Because importing is an expensive operation, by default Python does it just once per file, per process. Later import operations simply fetch the already loaded module object. Initialization code — As one consequence, because top-level code in a module file is usually executed only once, you can use it to initialize names once, but allow their state to change. As a demo, con

sider the file `init.py` in Example 23-2. Example 23-2. in `it.py`

```
python print('hello') flag = 1 # Initialize variable – just once!
```

In this example, the `print` and `=` statements run the first time the module is imported, and the variable `flag` is initialized at import time (recall from Chapter 17 that the REPL works like another module file here):

```
python $ python3 >>> import init # First import: loads and runs file's code hello
```

```
>>> init.flag # Assignment makes an attribute 1
```

Second and later imports don't rerun the module's code; they just fetch the already created module object from Python's internal modules table. Thus, the variable `flag` is not reinitialized:

```
python >>> init.flag = 2 # Change attribute in module >>> import init # Just fetches already loaded module >>> init.flag # Code wasn't rerun: attribute unchanged 2
```

Of course, sometimes you really want a module's code to be rerun on a subsequent import. You'll see how to do this with Python's `reload` function later in this chapter.

中文翻译

人们刚开始使用模块时最常问的问题之一就是："为什么我的 import 不生效了？"他们往往报告：第一次导入好好的，但之后在交互式会话（或程序运行）中的再次导入似乎毫无效果。事实上，这是刻意设计的——模块只在第一次 import 或 from 时被加载和运行，而且只有第一次。因为导入是开销很大的操作，默认情况下 Python 对每个文件、每个进程只做一次。之后的导入操作只是取回已经加载好的模块对象。初始化代码——由此带来的一个结果是：由于模块文件里的顶层代码通常只执行一次，你可以用它做一次性名字初始化，但允许这些名字的状态之后被改变。作为演示，请看示例 23-2 中的 init.py 文件。示例 23-2. init.py

```
python print('hello') flag = 1 # 初始化变量——只执行一次！
```

在这个示例中，print 和 = 语句在模块第一次被导入时执行，变量 flag 在导入时被初始化（回想第 17 章：这里的 REPL 就像另一个模块文件一样工作）：

```
python $ python 3 >>> import init # 首次导入：加载并运行文件代码hell
o >>> init.flag # 赋值产生一个属性1 第二次及之后的导入不会重新运行模块代码；它们只是从 Python 的内部模块表里取回已经创建好的模块对象。因此变量 flag 不会被重新初始化：python >>> init.flag = 2 # 修改模块中的属性>>> import init # 只是取回已加载的模块>>> init.flag # 代码没有重跑：属性没有变回原值2
```

当然，有些时候你真的希望在后续导入时重新运行模块代码。你会在本章后面看到如何用 Python 的 reload 函数做到这一点。

深度理解

·核心概念：模块每进程只加载运行一次。这是 Python 模块系统最容易被新手误解、却最根本的行为。

·底层实现：所谓"内部模块表"就是 `sys.modules` 字典（模块名 → 模块对象）。`import` 的第一件事就是查这张表：命中 → 直接返回缓存对象；未命中 → 才走"查找 → 编译 → 执行"的完整流程，然后把结果登记进表。`init.flag = 2` 之后再次 `import init`，由于 `sys.modules['init']` 已存在，代码不会重跑，`flag` 保持 2。

·设计原因：导入要读文件、编译字节码、执行顶层语句——昂贵。对大项目，一个模块可能被几十个文件 `import`；若每次 `import` 都重跑，轻则浪费、重则无限递归（模块 A `import` B，B `import` A）。

·实际问题：模块顶层正是放"一次性初始化"的地方——常量、默认配置、注册表、`print` 启动横幅等。而"改完文件 `reimport` 没反应"是新人开发时最大的困惑，答案就是：这不是 bug，是缓存。

·初学者误区：①以为再次 `import` 会拿到新代码（不会，必须 `reload`）；②以为每个文件 `import` 一次就重跑一次（错，全进程只有第一次真正跑）。

代码分析 (Example 23-2)

```
print('hello')  
flag = 1          # —
```

·解释器如何处理：`import init` 首次执行时，解释器编译 `init.py`，逐条运行 `print('hello')` 与 `flag = 1`，随后把模块对象连同属性 `flag` 存入 `sys.modules['init']`。第二次 `import init` 时，查找表直接命中，文件代码一行都不会再执行——所以 `flag` 不会被重置为 1。

·设计思想：模块既是"代码库"也是"单例状态容器" (singleton state holder)：顶层变量相当于全局单例的字段，第一次导入即初始化，之后可被任意使用者原地修改并共享。

23.7 导入是运行时赋值 (Imports Are Runtime Assignments)

英文原文

Just like `def`, `import` and `from` are executable statements, not compile-time declarations. They may be nested in `if` tests, to select among module options; appear in function defs, to be loaded only on calls (subject to the preceding note); be used in `try` statements, to provide defaults on errors; and so on. As an abstract example:

```
python if sometest: from moduleA import name else:
from moduleB import name
```

Wherever they appear, imports are not resolved or run until Python reaches them while executing your program. As one upshot, imported modules and names are not available until their associated `import` or `from` statements run. Changing mutables in modules — Also like `def`, `import` and `from` are implicit assignments: `import` assigns an entire module object to a single name. `from` assigns one or more names to objects of the same names in an

other module. Hence, everything you've already learned about assignment applies to module access, too. For instance, names copied with a `from` become references to shared objects; much like function arguments, reassigning a copied name has no effect on the module from which it was copied, but changing a shared mutable object through a copied name can also change it in the module from which it was imported. Consider the following file, `share.py`, in Example 23-3. Example 23-3. `share.py`

```
python x = 1 y = [1, 2]
```

When importing with `from`, we copy names to the importer's scope that initially share objects referenced by the module's names (again, the REPL serves as the importing module here):

```
python $ python3 >>> from share import x, y # Copy two names out
>>> x = 'hack' # Changes local x only
>>> y[0] = 'hack' # Changes shared mutable in place
>>> x, y # This module's x and y ('hack', ['hack', 2])
```

Here, `x` is not a shared mutable object, but `y` is. The names `y` in the importer and the imported modules both reference the same list object, so changing it from one place changes it in the other; continuing the REPL session:

```
python >>> import share # Get module name (from doesn't)
>>> share.x # share's x is not my x 1
>>> share.y # But we share a changed mutable ['hack', 2]
```

For more background on this, see Chapter 6. And for a graphical picture of what from assignments do with references, flip back to Figure 18-1 (function argument passing), and mentally replace "caller" and "function" with "imported" and "importer." The effect is the same, except that here we're dealing with names in modules, not functions. Assignment works the same everywhere in Python. Cross-file name changes — In the preceding example, the assignment to `x` in the interactive session changed the name `x` in that scope only, not the `x` in the file—there is no link from a name copied with from back to the file it came from. To really change a global name in another file, you must use `import`:

```
python $ python3 >>> from share import x, y # Copy
two names out >>> x = 23 # Changes my x only >>
> import share # Get module name >>> share.x = 23
# Changes x in other module This phenomenon was i
ntroduced in Chapter 17. Because changing variables
in other modules like this is a common source of con
fusion (and often a subpar design choice), we'll revisi
t this technique again later in this part of the book. S
ubtly, the change to y[0] in the prior session is differe
nt; it changes an object, not a name, and the name in
both modules references the same, changed object. T
his can be similarly subpar unless all module clients e
xpect it.
```

中文翻译

和 `def` 一样，`import` 与 `from` 也是可执行语句，而不是编译期声明。它们可以嵌套在 `if` 测试中，用来在多个模块选项间做选择；可以出现在函数 `def` 里，只在调用时加载（受前面那条注意的约束）；可以放进 `try` 语句，在出错时提供默认值；等等。举个抽象的例子：`python if sometest: from moduleA import name else: from moduleB import name` 无论出现在哪里，`import` 都要等 Python 执行到那一行时才会被解析和运行。由此带来的一个结果是：导入的模块和名字，在其对应的 `import` 或 `from` 语句运行之前是不可用的。修改模块中的可变对象——与 `def` 一样，`import` 和 `from` 也是隐式赋值：`import` 把整个模块对象赋给一个名字；`from` 把一个或多个名字赋值为另一个模块中同名的对象。因此，你学过的关于赋值的一切知识都适用于模块访问。例如，用 `from` 复制来的名字会成为共享对象的引用；和函数参数非常类似，重新给复制来的名字赋值不会影响原模块，但通过复制来的名字修改共享的可变对象，会同样改变它在原模块中的内容。请看示例 23-3 中的文件 `share.py`：示例 23-3. `share.py` `python x = 1 y = [1, 2]` 用 `from` 导入时，我们复制名字到导入方的作用域，这些名字最初共享着模块名字所引用的对象（同样地，这里的 REPL 充当导入方模块）：`python $ python3 >>> from share import x, y # 复制两个名字出来>>> x = 'hack' # 只修改本地的 x>>> y[0] = 'hack' # 就地修改共享的可变对象>>> x, y # 本模块的 x 和 y ('hack', ['hack', 2])` 这里 `x` 不是共享的可变对象，但 `y` 是。导入方和被导入模块里的名字 `y` 都引用同一个列表对象，所以从任何一处修改它，另一处也

会变；继续 REPL 会话：`python >>> import share # 取得模块名 (from 不会给) >>> share.x # share 的 x 不是我的 x`
`1 >>> share.y # 但我们共享一个被修改过的可变对象['hack', 2]` 关于此的更多背景参见第 6 章。想直观看到 `from` 赋值对引用做了什么，翻回图 18-1（函数参数传递），在脑中把“调用者（caller）”和“函数（function）”替换成“被导入方”和“导入方”即可。效果是一样的，只不过这里我们面对的是模块中的名字，而不是函数中的。赋值在 Python 的任何地方行为一致。跨文件修改名字——在上面的示例中，交互式会话里对 `x` 的赋值只改变了那个作用域中的名字 `x`，而没有改变文件中的 `x`——`from` 复制来的名字与它的来源文件之间没有任何联系。要真正改变另一个文件里的全局名字，你必须使用 `import`：`python $ python3 >>> from share import x, y # 复制两个名字出来 >>> x = 23 # 只修改我的 x >>> import share # 取得模块名 >>> share.x = 23 # 修改另一个模块中的 x` 这一现象在第 17 章介绍过。因为这样跨模块改变变量常常造成困惑（而且往往是不太好的设计选择），我们会在本书这一部分的后文再次讨论这种技术。微妙之处在于：之前会话中对 `y[0]` 的修改是不同的——它改变的是一个对象，而不是名字；两个模块里的名字引用的是同一个、被修改过的对象。除非所有模块的使用者都预期如此，否则这同样可能是欠妥的设计。

深度理解

·核心概念：`import/from` 是可执行赋值，不是编译期声明。它们出现的位置决定它们何时生效。

- 底层实现：条件导入在字节码层面就是一条普通的赋值分支——if sometest: 为真才执行 from moduleA import name, 此时才发生名字绑定；在这之前 name 未定义。所以"导入前用该名字"必然 NameError。
- 设计原因：动态语言的价值就在运行时决策——同一份代码可以按平台、按配置、按运行时探测选择不同实现（例如 try: import simplejson 失败则 import json）。
- 实际问题：这是写跨平台库的标准手法：if sys.platform == 'win32': import winimpl; 还有"可选依赖"模式（某库装了就用，没装就降级）。
- 初学者误区：以为 import 必须写在文件最顶部（那是惯例和最佳实践，不是语法要求）；以为"编译期声明"式的导入在 Python 存在——不存在。

代码分析 (Example 23-3: from 的复制引用语义)

```
x = 1
y = [1, 2]
```

```
$ python3
>>> from share import x, y      #
>>> x = 'hack'                  # x
>>> y[0] = 'hack'               #
>>> x, y                         # x y
('hack', ['hack', 2])
```

- 解释器如何处理：from share import x, y 执行时，解释器把 share.x（整数 1）和 share.y（列表对象 [1,2]）的引用复制到当前作用域。之后：
- x = 'hack': 是重新绑定当前作用域的名字 x，让 x 指向

新字符串；share.x 仍指向整数 1——两不相干。

- `y[0] = 'hack'`：是原地修改列表对象的元素，并没有重新绑定 `y`。当前作用域的 `y` 和 `share.y` 指向同一个列表对象，所以对象内容一变，两边同时可见 `['hack', 2]`。

- 设计思想：这就是 Python 引用语义（reference semantics）的统一性——与函数传参（图 18-1）完全同构：改名字（rebind）不出墙，改对象（mutate）全共享。一条铁律：`=` 换引用，`[下标]=/方法调用`改内容。

```
$ python3
>>> from share import x, y    #
>>> x = 23                    # x
>>> import share               #
>>> share.x = 23               # x
```

- 解释器如何处理：`share.x = 23` 是在模块对象的命名空间里重新绑定属性 `x`——这是“改到别人家”的唯一正道：必须拿到模块对象（`import` 是取模块对象的唯一途径，`from` 不给），再通过限定名赋值。而 `x = 23` 只改当前作用域。

- 设计思想：模块名就是“别人的命名空间的门牌号”。想动别人家的全局名？先 `import` 拿到门牌号，再用模块名.属性 = 值。不 `import` 就无法访问，`from` 拿到的只是“复印件”（引用），无法回写。

23.8 import 与 from 的等价性 (import and from Equivalence)

英文原文

Notice in the prior example that we have to execute an import statement after the from to access the shared module name at all. from only copies names from one module to another; it does not assign the module name itself. It may help to remember that, at least conceptually, a from statement like this one: `python from module import name1, name2` # Copy these two names out (only) is equivalent to this statement sequence: `python import module` # Fetch the module object `name1 = module.name1` # Copy names out by assignment `name2 = module.name2` `del module` # Get rid of the module name – here only. Like all assignments, the from statement creates new variables in the importer, which initially refer to objects of the same names in the imported file. Only the names are copied out, though, not the objects they reference, and not the name of the module itself. When we use the from ... form of this statement (`from module import ...`), the equivalence is the same, but all the top-level names in the module are copied over to the importing scope this way. Importantly, the first step of the from runs a normal import operation, with all the semantics outlined in the preceding chapter. As a result, the from always imports the entire module into memory if it has not yet been imported, regardless of how many names it copies out of the file. There is no way to load just part

of a module file (e.g., just one function), but because modules are bytecode in standard Python instead of machine code, the performance implications are generally negligible.

中文翻译

注意在前面的示例中，我们必须在 `from` 之后再执行一条 `import` 语句，才能访问到 `share` 这个模块名。`from` 只把名字从一个模块复制到另一个模块；它不会给模块名本身赋值。记住下面这一点会有帮助：至少从概念上讲，一条这样的 `from` 语句：`python from module import name1, name2` # 只复制这两个名字出来等价于下面这一组语句：`python import module` # 取得模块对象 `name1 = module.name1` # 通过赋值把名字复制出来 `name2 = module.name2` `del module` # 丢弃模块名——只是在这里和所有赋值一样，`from` 语句在导入方创建新变量，这些变量最初引用被导入文件中同名对象。复制出来的只有名字，而不是它们引用的对象，更不是模块本身的名字。当我们使用 `from` 的 `\` 形式 (`from module import \`) 时，等价关系相同，只是模块中所有顶层名字都这样被复制到导入方作用域。重要的是，`from` 的第一步执行的是一次普通的导入操作，带有前一章讲过的全部语义。因此，如果模块尚未被导入，无论 `from` 从文件里复制出多少个名字，它总是把模块完整加载进内存。没有办法只加载模块文件的一部分（比如只加载一个函数），但由于标准 Python 中模块是字节码 (bytecode) 而非机器码，性能影响通常可以忽略。

深度理解

- 核心概念：from 的"展开式"——import + 逐个赋值 + del 模块名。这条等价关系是理解 from 一切行为的钥匙。
- 底层实现：展开式里的 del module 解释了为什么 from 之后模块名未定义；而 name1 = module.name1 解释了为什么只复制引用、不复制对象、更不建立"回链"。from 的真实字节码就是这个展开式的优化版本。
- 设计原因：把 from 定义为 import 的"语法糖扩展"，保证两条语句共享同一套导入管线，避免两套语义分裂。
- 实际问题：性能问题由此化解——既然 from 也必须加载整个模块，那么"为了快而用 from"毫无必要；同时"部分加载"（按需只取函数）在标准 Python 中不存在。
- 初学者误区：以为 from 只把用到的东西装进内存（不是，整个模块都会执行）；以为 from 之后还能用模块名（展开式里模块名已被 del）。

23.9 from 语句的潜在陷阱 (Potential Pitfalls of the from Statement)

英文原文

One downside of the from statement is that it makes the meaning of a variable more obscure: name is less useful to the reader than module.name, and may req

uire a search for the from that loaded it. Because of this, some Python users recommend coding `import` instead of `from` most of the time. Like most advice, though, this doesn't always make sense. `from` is commonly and widely used, without dire consequences. Moreover, it's convenient not to have to type a module's name every time you wish to use one of its tools, especially when working in the REPL. It is true that the `from` statement has the potential to corrupt namespaces, at least in principle—if you use it to import variables that happen to have the same names as existing variables in your scope, your variables will be silently overwritten. This problem doesn't occur with the `import` statement because you must always go through a module's name to get to its contents: `module.attr` will not clash with a variable named `attr` in your scope. As long as you understand that this can happen when using `from`, though, this isn't a concern in practice: assigning a variable with `from` has the same effect as any other assignment in your code. On the other hand, the `from` statement has legitimate issues when used in conjunction with the `reload` call, as imported names might reference prior versions of objects. Moreover, the `from ... import` really can trash namespaces and make code difficult to understand, especially when applied to more than one file—in this case, there is no way to tell which module a name came from, short of searching external files. In effect, the `from ... import` coll

apses one namespace into another, and so defeats the namespace partitioning purpose of modules. We'll save demos of these issues for "Module Gotchas" (at the end of this part of the book), and meet tools that can minimize the damage with data hiding in Chapter 25. Probably the best real-world advice here is to generally prefer `import` to `from` for simple modules; to explicitly list the variables you want in most `from` statements; and to limit the `from` form to just one per file, or the REPL's throw-away code. That way, any names not called out by attribute qualification or `from` lists can be assumed to live in the sole module imported by the `from`. Some care is required when using the `from` statement, but armed with a little knowledge, most programmers find it to be a convenient way to access modules. When `import` is required — The only time you really must use `import` instead of `from` is when you must use the same name defined in two different modules. For example, imagine that two files define the same name differently, as in this abstract snippet:

```
python # M.py def func(): ...do something... # N.py def func(): ...do something else... If you must use both versions of this name in your program, the from statement will fail—you can have only one assignment to the name in your scope:
```

```
python # O.py from M import func from N import func # This overwrites the one we fetched from M func()
```

) # Calls N.func only!

An import will work here, though, because including the name of the enclosing module makes the two names unique:

```
python # O.py import M, N # Get the whole modules, not their names
M.func() # We can call both names now
N.func() # The module names make them unique
```

This case is unusual enough that you're unlikely to encounter it very often in practice. If you do, though, import allows you to avoid the name collision. Another way out of this dilemma is using the as extension, a renaming tool that we'll cover fully in Chapter 25 but is simple enough to introduce here:

```
python # O.py from M import func as mfunc # Rename uniquely with "as"
from N import func as nfunc
mfunc(); nfunc() # Call one or the other
```

The as extension works in both import and from as a simple renaming tool (it can also be used to give a shorter synonym for a long module name in import); more on this form in Chapters 24 and 25.

中文翻译

from 语句的一个缺点是它让变量的含义变得更模糊：name 对读者来说不如 module.name 有用，读者可能要去寻找是哪个 from 把它加载进来的。正因为如此，有些 Python 用户建议大多数时候用 import 而不是 from。不过和大多数建议一样，这并不总是有道理。from 被普遍而广泛地使

用着，并没有带来什么严重后果。何况，每用一次工具就得敲一遍模块名确实麻烦，尤其是在 REPL 里工作时。from 语句确实有污染命名空间的潜在可能——至少原则上如此：如果你用它导入的变量恰好与你作用域中已有的变量同名，你的变量会被静默覆盖。这个问题不会发生在 import 语句上，因为你总是要通过模块名才能取到内容：module.attr 不会与你作用域里名为 attr 的变量冲突。不过，只要你明白使用 from 时可能发生这种情况，实践中这就不算个事：用 from 赋值变量，效果和你在代码里做任何其他赋值是一样的。另一方面，from 语句与 reload 调用配合使用时存在真正的问题——导入的名字可能引用的是旧版本的对象。此外，from \ 形式真的能摧毁命名空间、让代码难以理解，尤其是当它作用于不止一个文件时——这种情况下，除了去外部文件里搜索，你根本无法判断某个名字来自哪个模块。实际上，from \ 把一个命名空间塌缩进另一个，从而瓦解了模块“划分命名空间”的意义。这些问题的演示我们留到本书这一部分的“模块陷阱 (Module Gotchas)”里，第 25 章则会介绍通过数据隐藏来减少 from \ 危害的工具。这里最实用的建议大概是：对简单模块，总体上优先用 import 而不是 from；大多数 from 语句里要明确列出你想要的变量；把 from \ 限制为每文件至多一次，或者只在 REPL 的临时代码里用。这样一来，任何没有通过属性限定或 from 列表点名的名字，都可以假定它存在于那个被 from \ 导入的唯一模块中。使用 from 语句确实需要一些小心，但掌握了这点知识后，大多数程序员都会觉得它是访问模块的便捷方式。何时必须用 import——唯一真正必须用 import 而不用 from 的时刻，是当你必须同时使用定义在两个不同模块中的同名名字时。例如，设想两个文件对同名

名字给出了不同定义，就像下面这段抽象代码：python # M.py def func(): ...做一些事情... # N.py def func(): ...做另一些事情... 如果你的程序必须同时使用这个名字的两个版本，from 语句会失败——你的作用域里一个名字只能有一个赋值：python # O.py from M import func from N import func # 这会覆盖我们从 M 取来的那个func() # 只调用 N.func! 而 import 在这里可以工作，因为带上所在模块的名字，两个名字就唯一了：python # O.py import M, N # 取得整个模块，而不是它们的名字M.func() # 现在两个名字都能调用N.func() # 模块名使它们互不冲突这种情况相当少见，实践中不太常遇到。但如果真遇到了，import 能让你避开名字冲突。摆脱这个困境的另一个办法是用 as 扩展，一种重命名工具——第 25 章会完整介绍它，但在这里介绍它已经足够简单：python # O.py from M import func as mfunc # 用 "as" 唯一地重命名from N import func as nfunc mfunc(); nfunc() # 想调哪个调哪个 as 扩展在 import 和 from 中都是简单的重命名工具（在 import 里也可以给冗长的模块名起个简短别名）；更多相关内容见第 24、25 章。

深度理解

·核心概念：from 的三大风险——①来源模糊（可读性）；②静默覆盖（命名空间污染）；③与 reload 配合时引用过期对象。from \ 是三者的加严重化版。

·底层实现：静默覆盖的机制很直白：from 本质是赋值，而赋值就是无条件覆盖当前作用域里的同名绑定，不报任何警告。reload 的问题在于：from 复制的是"当时"的引用，reload 更新的是模块对象里的属性绑定，你手里的旧

引用纹丝不动。

·设计原因：模块的核心价值就是“命名空间分区”——把不同来源的名字隔开。from \ 把这个分区整个取消掉，所以它是被谨慎对待的语法。

·实际问题：这就是为什么企业代码规范（PEP 8 也提倡）要求显式导入；from x import 在 lint 工具（flake8、ruff）里默认就是错误级别警告。

·初学者误区：①以为 from 导致的问题罕见（同名覆盖很常见，尤其 from os import 会覆盖 open、listdir 等）；②以为“必须用 import”的场景很多（其实只有两个模块同名这一个硬性场景）；③不知道 as 别名既能救场又能简化长模块名。

代码分析（Example: O.py 三种写法）

```
# O.py
from M import func
from N import func      # M
func()                  # N.func
```

·解释器如何处理：第二条 from 执行时，当前作用域里已有的 func 绑定被新的 N.func 引用无条件替换，M 的函数对象失去访问路径（仅当没有其他引用时才可能被垃圾回收）。这不是报错，而是静默的覆盖——最危险的一类 bug。

```
# O.py
import M, N              #
M.func()                 #
N.func()                 #
```

·解释器如何处理：import 只绑定 M、N 两个名字，各自的 func 属性通过限定访问取用——名字空间天然隔离，冲突不可能发生。

```
# 0.py
from M import func as mfunc    # "as"
from N import func as nfunc
mfunc(); nfunc()               #
```

·解释器如何处理：as 让解释器把复制出来的引用绑定到别名上，等价于 mfunc = M.func。两条绑定互不重叠，兼顾了 from 的便利与 import 的安全。

·设计思想：三种写法对应三个权衡档位——裸 from（省事但易撞）、import（安全但繁琐）、from + as（折中方案）。

23.10 模块命名空间 (Module Namespaces)

英文原文

Modules are probably best understood as packages of names—i.e., places to define names you want to make visible to the rest of a system. Technically, modules usually correspond to files, and Python creates a module object to contain all the names assigned in a module file. But in simple terms, modules are just namespaces (places where names are created), and the names that live in a module are its attributes. This section expands on the details behind this model.

中文翻译

模块最好被理解为一包名字 (packages of names) —— 也就是说，是定义那些你希望系统其余部分可见的名字的地方。从技术上讲，模块通常对应文件，Python 会创建一个模块对象来容纳模块文件中赋值的所有名字。但用简单的话说：模块就是命名空间 (namespaces，创建名字的场所)，住在模块里的名字就是它的属性。本节将展开这个模型背后的细节。

深度理解

- 核心概念：模块 = 命名空间。这是全章最抽象也最重要的一句话：模块的价值不在"文件"这个物理载体，而在"名字的分區"这个逻辑作用。
- 底层实现：Python 为每个模块创建一个 module 对象，内部用一个字典存所有名字——这就是"命名空间"的物质形态。
- 设计原因：把名字按文件分区，是大规模协作的基石：两个人写两个文件，同名变量互不干扰，需要时再显式互通。
- 实际问题：团队协作、第三方库共存 (requests 里的 get 不会撞上你的 get)、重构时的局部影响——都靠这层分区。
- 初学者误区：以为"命名空间"是玄学概念。记住物质形态即可：它就是模块对象里的那本名字字典。

23.10.1 文件如何生成命名空间 (How Files Generate Namespaces)

英文原文

We've seen that files morph into namespaces, but how does this actually happen? The short answer is that every name that is assigned a value at the top level of a module file (i.e., not nested in a function or class body) becomes an attribute of that module. For instance, given an assignment statement such as `X = 1` at the top level of a module file `M.py`, the name `X` becomes an attribute of `M`, which we can refer to from outside the module as `M.X`. The name `X` also becomes a global variable to other code inside `M.py`, but we need to firm up the relationship of module loading and scopes to understand why:

- Module statements run on the first import. The first time a module is imported anywhere in a system, Python creates an empty module object and executes the statements in the module file one after another, from the top of the file to the bottom.
- Top-level assignments create module attributes. During an import, statements at the top level of the file not nested in a `def` or `class` that assign names (e.g., `=`, `def`) create attributes of the module object. Names assigned by these statements are stored in the module's namespace.
- Module namespaces are dictionaries. Module namespaces created by imports

may be accessed through the built-in dict dictionary attribute of module objects, and may be inspected with the dir function. dir is the same as the sorted keys of dict for modules, but includes inherited names for classes.- Modules are a single scope (local is global). As we saw in Chapter 17, names at the top level of a module follow the same scope rules as names in a function, but the local and global scopes are the same—or, more formally, they follow the LEGB scope rule presented in Chapter 17, but without the L and E lookup layers.- Module scopes outlive imports. A module's global scope becomes an attribute dictionary of a module object after the module has been loaded. Unlike function scopes, where the local namespace exists only while a function call runs, a module file's scope lives on after the import, providing a source of tools to importers. Here's a demonstration of these ideas. Suppose we create the module file in Example 23-4 with a text editor, and call it spaces.py. Example 23-4. spaces.py

```
python print('starting to load...') import sys var = 23
def func(): pass class klass: pass print('done loading.')
```

The first time this module is imported (or run as a program), Python executes its statements from top to bottom. Some statements create names in the module's namespace as a side effect, but others do actual work while the import is going on. For instance, the two print statements in this file execute at import time:

```
python >>> import spaces starting to load... done loading.
```

Once the module is loaded, its scope becomes an attribute namespace in the module object we get back from import. We can then access attributes in this namespace by qualifying them with the name of the enclosing module:

```
python >>> spaces.sys <module'sys' (built-in)>
```

```
>>> spaces.var 23
```

```
>>> spaces.func <function func at 0x101ce7b00> >
```

```
>> spaces.klass <class'spaces.klass'> Here, sys, var, func, and klass were all assigned while the module's statements were being run, so they are attributes after the import. We'll study classes in Part VI, but notice the sys attribute—import statements assign module objects to names, and any type of assignment to a name at the top level of a file generates a module attribute.
```

中文翻译

我们已经看到文件会“变身”为命名空间，但这究竟是怎么发生的？简短的回答是：模块文件顶层（即不在函数或类体内）被赋值的每一个名字，都会成为该模块的一个属性。例如，在模块文件 `M.py` 顶层有一条赋值语句 `X = 1`，名字 `X` 就成为 `M` 的一个属性，我们可以从模块外部以 `M.X` 引用它。同时 `X` 也是 `M.py` 内部其他代码的全局变量，但要理解其中缘由，我们需要把模块加载与作用域的关系讲扎实： -

模块语句在首次导入时运行。系统中任何地方第一次导入某模块时，Python 会创建一个空的模块对象，并从上到下逐条执行模块文件中的语句。

- 顶层赋值创建模块属性。导入期间，文件顶层不在 `def` 或 `class` 内的、对名字进行赋值的语句（例如 `=`、`def`）会创建模块对象的属性。这些语句赋值的名字被存入模块的命名空间。
- 模块命名空间是字典。导入产生的模块命名空间，可以通过模块对象的内置 `dict` 字典属性访问，也可以用 `dir` 函数检查。对模块而言，`dir` 与 `dict` 的排序键完全相同；但对类而言它还包括继承来的名字。
- 模块是单一作用域（局部即全局）。如第 17 章所见，模块顶层的名字遵循与函数中名字相同的作用域规则，但局部作用域和全局作用域是同一个——或者更正式地说，它们遵循第 17 章介绍的 LEGB 作用域规则，只是没有 `L` 和 `E` 这两层查找。
- 模块作用域比导入活得更久。模块加载后，它的全局作用域就变成模块对象的属性字典。与函数作用域不同（函数调用的局部命名空间只在调用期间存在），模块文件的作用域在导入后依然长存，为导入方提供源源不断的工具。下面演示一下这些观点。假设我们用文本编辑器创建示例 23-4 的模块文件，并命名为 `spaces.py`。示例 23-4. `spaces.py`

```
python print('starting to load...') import sys var = 23 def func(): pass class klass: pass print('done loading.')
```

这个模块第一次被导入（或作为程序运行时），Python 从上到下执行它的语句。有些语句作为副作用在模块命名空间中创建名字，有些则在导入进行过程中干实事。例如，文件中的两条 `print` 语句在导入时就会执行：

```
python >>> import spaces starting to load... done loading.
```

模块一旦加载完成，它的作用域就变成我们从 `import` 拿回的模块对象上的属性命名空间。之后我们可以用所在模

块名限定这些属性来访问它们：`python >>> spaces.sys`
`<module'sys' (built-in)> >>> spaces.var 23 >>> spaces.func`
`<function func at 0x101ce7b00> >>> spaces.klass`
`<class'spaces.klass'>` 这里，`sys`、`var`、`func`、`klass` 都是在模块语句运行期间被赋值的，所以导入后它们都是属性。我们会在第六部分学习类，但请注意 `sys` 这个属性——`import` 语句把模块对象赋给名字，而文件顶层任何形式的赋值都会生成模块属性。

深度理解

·核心概念：文件 → 命名空间的五条铁律：首导即跑、顶层赋值即属性、命名空间是字典、单作用域（局部=全局）、作用域长存。

·底层实现：第五条尤其关键——函数作用域是“栈帧临时簿”，调用结束就销毁；模块作用域是“永驻的属性字典”，保存在 `sys.modules` 里供所有导入方持续访问。所以模块里的全局变量是天然的“进程级共享状态”。

·设计原因：局部即全局（没有 L、E 层）意味着模块顶层不需要 `global` 声明——顶层代码本身就在全局作用域里；而函数里改模块变量才需要 `global`（如 `lex1.py` 示例）。

·实际问题：理解这五条，你就能回答“为什么 `import` 一个模块会打印东西”（首导即跑）、“为什么模块里随便写的变量别人都能看到”（顶层赋值即属性）。

·初学者误区：①以为模块里函数内部的局部变量也会变成属性（不会，只有顶层赋值才算）；②以为作用域是“运行

时动态"的（不是，作用域由文件里的物理位置静态决定）；③看到 `spaces.sys` 惊讶于"模块里居然有 `sys` 属性"——`import` 也是赋值，任何赋值都生成属性。

代码分析 (Example 23-4)

```
print('starting to load...')
import sys
var = 23
def func(): pass
class klass: pass
print('done loading.')
```

```
>>> import spaces
starting to load...
done loading.
```

- 解释器如何处理：`import spaces` 首次执行时，解释器创建空的模块对象，然后逐条编译执行：
- 两条 `print` 立即输出（所以导入会"有声音"）；
- `import sys` 把 `sys` 模块对象绑定到名字 `sys`——于是 `sys` 也成为 `spaces` 的属性；
- `var = 23`、`def func`、`class klass` 分别把名字写入模块的 `dict`；
- 执行完毕后，`spaces.sys`、`spaces.var`、`spaces.func`、`spaces.klass` 全部可访问。
- 设计思想：模块加载就是"按顺序执行一遍文件"，没有魔法。凡是顶层写下的名字，无论来自 `print`、`import`、赋值、`def` 还是 `class`，一视同仁成为属性——这正是"赋值即导出"哲学的完整演示。

23.10.2 命名空间字典: dict (Namespace Dictionaries: dict)

英文原文

In fact, internally, module namespaces are stored as dictionary objects. These are just normal dictionaries with all the usual methods. When needed, we can access a module's namespace dictionary through the module's `dict` attribute—for instance, to write tools that list module content generically, as we will in Chapter 25. Continuing the prior section's example:

```
python >>> spaces.dict.keys() dictkeys(['name','doc','package','loader','spec','file','cached','builtins','sys','var','func','klass'])
```

The names we assigned in the module file become dictionary keys internally, so some of the keys here reflect top-level assignments in our file. The value of `var`, for example, can be had two ways (though the first is normal):

```
python >>> spaces.var, spaces.dict['var'] (23, 23)
```

Python also adds some useful names in the module's namespace. For instance, `file` gives the name of the file from which the module was loaded (useful to find resources bundled with code), and `name` gives its name as known to importers (without the `.py` extension and directory path):


```
python >>> spaces.name, spaces.file ('spaces','/Users  
/me/.../LP6E/Chapter23/spaces.py')
```

To see just the names your code assigns, filter out double-underscore names as we did in Chapter 15's `dir` coverage and Chapter 17's built-in scope coverage, but this time with a generator expression. We can also omit keys because dictionaries generate keys automatically, and the `dir` built-in for modules is just the sorted keys list of dict:

```
python >>> list(name for name in spaces.dict if not name.startswith('__')) ['sys','var','func','klass']
```

```
>>> dir(spaces) == sorted(spaces.dict) True
```

As another lesser-used alternative, Python's `vars` built-in function simply fetches the dict of a module passed to it (an option, perhaps, after you've written enough Python code to wear out your keyboard's underscore key?):

```
python >>> vars(spaces) == spaces.dict True
```

```
>>> dir(spaces) == sorted(vars(spaces)) True
```

See Chapter 7 for other `vars` roles. You'll see similar dict dictionaries on class-related objects in Part VI too. In all cases, attribute fetch is similar to dictionary indexing, though only the former kicks off inheritance in classes.

中文翻译

事实上，模块命名空间在内部就是字典对象。它们就是普通的字典，拥有所有常用方法。需要时，我们可以通过模块的 `dict` 属性访问模块的命名空间字典——例如，用来编写通用地列出模块内容的工具，第 25 章我们会这么做。继续上一节的示例：`python >>> spaces.dict.keys()` `dictkeys(['name','doc','package','loader','spec','file','cached','builtins','sys','var','func','klass'])` 我们在模块文件中赋值的名字，内部成为字典的键，所以这里的部分键反映了我们文件里的顶层赋值。例如 `var` 的值有两种取法（虽然第一种是常规用法）：`python >>> spaces.var, spaces.dict['var']` (`23, 23`) Python 还会在模块命名空间里添加一些有用的名字。例如 `file` 给出模块加载自哪个文件（对寻找源代码打包的资源很有用），`name` 给出模块在导入方那里的名字（不带 `.py` 扩展名和目录路径）：`python >>> spaces.name, spaces.file('spaces','/Users/me/.../LP6E/Chapter23/spaces.py')` 只想看你自己的代码赋了哪些名字，可以像第 15 章的 `dir` 专题和第 17 章的内置作用域专题那样过滤掉双下划线名字，只不过这次用生成器表达式。我们也可以省略 `keys`，因为字典会自动生成键；而模块的 `dir` 内置函数就是 `dict` 的排序键列表：`python >>> list(name for name in spaces.dict if not name.startswith('_'))` `['sys','var','func','klass']` `>>> dir(spaces) == sorted(spaces.dict)` `True` 另一个较少用的替代方案是 Python 的 `vars` 内置函数，它只是取出传给它的模块的 `dict`（也许等你写过足够多的 Python 代码、把键盘的下划线键敲坏了以后，这算个备选项？）：`python >>> vars(spaces) == spaces.dict` `True`

>>> dir(spaces) == sorted(vars(spaces)) True vars 的其他角色见第 7 章。第六部分你会看到类相关对象上也有类似的 dict 字典。在所有情况下，属性获取都与字典索引类似，只不过前者会在类中触发继承。

深度理解

·核心概念：模块命名空间 = 一本可翻阅的字典。dict 是窥探模块内部的"后门"；name、file 等是解释器注入的元信息。

·底层实现：spaces.var 在底层几乎就是 spaces.dict['var']——属性访问与字典索引同构。键里除了你的名字，还有解释器塞进去的系统名字：name（模块名）、file（源文件路径）、doc（文档字符串）、package、loader、spec（导入系统元数据）、cached（.pyc 路径）、builtins（内置函数所在的命名空间）。

·name 的两种状态（本章后文与全书的常用法）：当模块被 import 时，name 是模块名（如'spaces'）；当文件作为顶层脚本直接运行时，Python 会把它的 name 设为'main'。所以 if name == 'main': 判断的就是"这个文件是被导入还是被直接运行"——这是让一个文件"既能当模块、又能当程序"的标准开关。

·设计原因：把命名空间做成普通字典，意味着一切字典操作（遍历、过滤、判断包含）都能直接作用于模块内容——这让"内省（introspection）"和元编程成为可能。

·实际问题：dir(module) 一键查看模块所有公开名；插件框架可以扫描模块里所有函数并注册；file 用于定位资源

文件（如配置文件、模板）。

·初学者误区：①以为 dict 里只有自己写的名字（一大堆 dunder 系统名会让 spaces.dict 看起来很"脏"）；②以为 dir() 与 dict 完全相同（对模块是，对类不是——类还含继承名字）；③不知道 vars() 就是 dict 的别名。

代码分析 (dict 系列演示)

```
>>> spaces.__dict__.keys()
dict_keys(['__name__', '__doc__', '__package__', '__loader__', '__...
['__file__', '__cached__', '__builtins__', 'sys', 'var', 'func', 'k...
```

·解释器如何处理：dict 返回模块命名空间字典本身；.keys() 是普通字典的视图对象 (dictkeys)。键的集合 = 解释器注入的系统名 + 你自己赋值产生的名字。

```
>>> spaces.var, spaces.__dict__['var']
(23, 23)
```

·解释器如何处理：spaces.var 被编译成"取 spaces 对象的 var 属性"，内部正是对命名空间字典的索引操作；所以两种写法结果一致。属性语法多出的能力：在类对象上会触发继承链查找。

```
>>> list(name for name in spaces.__dict__ if not name.startswith...
['sys', 'var', 'func', 'klass']
```

·解释器如何处理：生成器表达式遍历字典（默认遍历键），startswith('') 过滤掉双下划线系统名，剩下的恰好是你亲手赋的名字。这是"看一个模块到底定义了啥"的标准手法。

```
>>> dir(spaces) == sorted(spaces.__dict__)
True
```

·解释器如何处理：dir() 对模块就是取 dict 键并排序，所以两者相等。对类则不同：dir 会沿 MRO（方法解析顺序）包含继承的成员。

·设计思想：命名空间即字典 → 内省零成本 → 元编程（动态列出、动态调用、动态注入）成为可能。这是 Python "万物皆对象、对象皆可查"哲学的集中体现。

23.10.3 属性名限定 (Attribute Name Qualification)

英文原文

Speaking of attribute fetch, now that you're becoming more familiar with modules, we should firm up the notion of name qualification more formally too. In Python, you can access the attributes of any object that has attributes using the qualification (a.k.a. attribute fetch) syntax `object.attribute`. Qualification is really an expression that returns the value assigned to an attribute name associated with an object. For example, the expression `spaces.sys` in the previous example fetches the value assigned to `sys` in `spaces`. Similarly, if we have a built-in list object `L`, `L.append` returns the append method object associated with that list. It's important to keep in mind that attribute qualification has nothing to do with the scope rules we studied in Chapter 17; it's an independent concept. When you use qualification to access names, you give Python an explicit object from which to fetch the specified names. T

he LEGB scope rule applies only to bare, unqualified names; it may be used for the leftmost name in a qualification path, but later names after dots search specific objects instead. This distinction may seem blurred by the fact that module scopes morph into attributes at the end of an import, but thereafter, names are part of a module object. For reference, here are the full rules for name resolution in Python:- Simple variables : X means search for the name X in the current scopes (following the LEGB rule of Chapter 17).- Qualification: X.Y means find X in the current scopes, then search for the attribute Y in the object X (not in scopes).- Qualification paths: X.Y.Z means look up the name Y in the object X, then look up Z in the object X.Y.- Generality: Qualification works on all objects with attributes: modules, classes, C extension types, etc. As noted earlier, in Part VI you'll see that attribute qualification means a bit more for classes—it's also the place where inheritance happens. In general, though, the rules outlined here apply to all names in Python.

中文翻译

既然谈到了属性获取，而且你现在对模块越来越熟悉了，我们也应该把名字限定（name qualification）的概念正式地夯实一下。在 Python 中，你可以用限定（qualification，也叫属性获取 attribute fetch）语法 `object.attribute` 访问任何拥有属性的对象的属性。限定本质上是一个表达式，返回与某个对象关联的、赋给某个属性名的值。例如，前面

例子中的表达式 `spaces.sys` 取出赋给 `spaces` 中 `sys` 的值。类似地，如果我们有一个内置的列表对象 `L`，`L.append` 会返回与那个列表关联的 `append` 方法对象。重要的是要记住：属性限定与第 17 章学习的作用域规则毫无关系；它是一个独立的概念。使用限定访问名字时，你给了 Python 一个明确的对象，让 Python 从这个对象上取指定的名字。LEGB 作用域规则只适用于裸的、未限定的名字；它可以用于限定路径中最左边的名字，但点号之后的那些名字是在具体对象里查找的，而不是在作用域里。“模块作用域在导入结束时变成属性”这一事实可能会模糊这种区分，但自此以后，名字属于模块对象的一部分。作为参考，以下是 Python 名字解析的完整规则：

- 简单变量：`X` 表示在当前作用域中查找名字 `X`（遵循第 17 章的 LEGB 规则）。
- 限定：`X.Y` 表示先在当前作用域找到 `X`，然后在对象 `X` 中查找属性 `Y`（不是在作用域中）。
- 限定路径：`X.Y.Z` 表示先在对象 `X` 中查找名字 `Y`，再在对象 `X.Y` 中查找 `Z`。
- 通用性：限定适用于所有拥有属性的对象：模块、类、C 扩展类型等等。如前所述，在第六部分你会看到属性限定对类意味着更多——继承也发生在那里。但总体而言，这里列出的规则适用于 Python 中的所有名字。

深度理解

- 核心概念：限定与作用域是两套独立的机制。裸名字走 LEGB（作用域）；带点名字先走 LEGB 找最左对象，然后“进入对象内部”找属性。
- 底层实现：`X.Y.Z` 的求值过程是：①在当前作用域查 `X`（唯一走 LEGB 的一步）；②对 `X` 做属性查找得到对象，记作临时对象 `A`；③对 `A` 做属性查找得到 `Y` 的对象 `B`；④

对 B 做属性查找得到 Z。每一步都是 `object.getattribute` 的调用（类对象还会涉及 MRO 继承链）。

·设计原因：把两种查找分开，才可能做到“模块里有个名字叫 X，你函数里也有个局部变量 X，两者互不干扰”——作用域管裸名，点号管对象属性。

·实际问题：调试时遇到 `NameError`（裸名没找到）与 `AttributeError`（对象上没这属性）是两种不同的错误，修复思路完全不同。

·初学者误区：以为 `module.attr` 也走作用域规则；以为点号路径的每一步都在当前作用域里查。记住：只有第一个点号前的名字走作用域，之后全是对象属性。

23.11 导入与作用域 (Imports Versus Scope s)

英文原文

As we've seen, it is never possible to access names defined in another module file without first importing that file. That is, you never automatically get to see names in another file, regardless of the structure of imports or function calls in your program. A variable's meaning is always determined by the locations of assignments in your source code, and attributes are always requested of an object explicitly. For example, consider the following two simple modules. The first, `lex1.py` in Example 23-5, defines a variable `X` global to

code in its file only, along with a function that changes the global X in this file. Example 23-5. lex1.py

```
python X = 88 # My X: global to this file only
def f():
    global X # Change this file's X
```

X = 99 # Cannot see names in other modules

The second module, lex2.py in Example 23-6, defines its own global variable X and imports and calls the function in the first module. Example 23-6. lex2.py

```
python X = 11 # My X: global to this file only
import lex1 # Gain access to names in lex1
lex1.f() # Sets lex1.X, not this file's X
```

print(X, lex1.X)

When run, lex1.f changes the X in lex1, not the X in lex2. The global scope for lex1.f is always the file enclosing it, regardless of which module it is ultimately called from:

```
python $ python3 lex2.py 11 99
```

In other words, import operations never give upward visibility to code in imported files—an imported file cannot see names in the importing file. More formally:

- Functions can never see names in other functions unless they are physically enclosing.
 - Module code can never see names in other modules unless they are explicitly imported.
- Such behavior is part of the lexical scoping notion—in Python, the scopes surrounding a piece of code are completely determined by the code's physical position in your file. Scopes are never in

fluenced by function calls or module imports. Some languages act differently and provide for dynamic scoping, where scopes really may depend on runtime calls. This tends to make code trickier, though, because the meaning of a variable can differ over time. In Python, scopes more simply correspond to the text of your program.

中文翻译

正如我们所见，不先导入一个模块文件，就永远不可能访问其中定义的名字。也就是说，无论程序里的导入结构或函数调用结构如何，你永远不会自动看到另一个文件里的名字。变量的含义总是由你的源代码中赋值语句的位置决定，属性也总是向某个对象显式索取的。例如，考虑下面两个简单的模块。第一个，示例 23-5 的 `lex1.py`，定义了一个仅对其文件内代码全局的变量 `X`，以及一个修改本文件全局 `X` 的函数。示例 23-5. `lex1.py` `python X = 88 # 我的 X: 只对本文件全局`
`def f(): global X # 修改本文件的 X`
`X = 99 # 看不到其他模块里的名字`
第二个模块，示例 23-6 的 `lex2.py`，定义了自己的全局变量 `X`，并导入、调用了第一个模块中的函数。示例 23-6. `lex2.py` `python X = 11 # 我的 X: 只对本文件全局`
`import lex1 # 获得 lex1 中名字的访问权`
`lex1.f() # 设置的是 lex1.X, 不是本文件的 X`
`print(X, lex1.X)`
运行时，`lex1.f` 修改的是 `lex1` 中的 `X`，而不是 `lex2` 中的 `X`。`lex1.f` 的全局作用域永远是包裹它的那个文件，无论它最终是从哪个模块被调用的：`python $ python3 lex2.py 11 99` 换句话说，导入操作绝不会给被导入文件中的代码“向上看”的能力——被导入的文件看不到导入方文件里的名

字。更正式地说：- 函数永远看不到其他函数中的名字，除非它们在物理上互相嵌套。- 模块代码永远看不到其他模块中的名字，除非它们被显式导入。这种行为是词法作用域（lexical scoping）概念的一部分——在 Python 中，围绕一段代码的作用域完全由代码在文件中的物理位置决定。作用域永远不会受函数调用或模块导入的影响。有些语言行为不同，提供动态作用域（dynamic scoping），其作用域真的可能取决于运行时调用。但这往往让代码更棘手，因为变量的含义可能随时间改变。在 Python 中，作用域更简单地对应着你程序的文本。

深度理解

- 核心概念：作用域 = 代码的物理位置决定的"领地"，与调用关系无关。lex1.f 的全局作用域永远是 lex1.py——即使从 lex2.py 调用它。
- 底层实现：函数对象在定义时就把自己的全局命名空间（func.globals）钉死为定义所在模块的字典。调用 lex1.f() 时，函数内 global X 修改的是 lex1.dict['X']，与 lex2 的 X 毫无瓜葛。所以输出 11 99：lex2 的 X 仍是 11，lex1 的 X 变成 99。
- 设计原因：词法作用域让程序"读文本即知含义"——同一条代码在任何地方执行含义不变，可预测、可静态分析。动态作用域（如早期 Lisp、Perl 的局部变量）则会让同一个变量在不同调用栈下含义不同，调试是噩梦。
- 实际问题：这就是为什么"共享状态"必须显式：要么 import 后限定访问，要么把可变对象传参。想让模块 A 改模块 B 的变量？只有 B.x = ... 这一条路。

·初学者误区：以为"我调用了 lex1 的函数，它就能看到我的变量"；以为 import 是"双向打通"。实际是：导入只给导入方"向下看"的能力，被导入方永远不知道谁导入了它。

23.11.1 命名空间嵌套 (Namespace Nesting)

英文原文

Finally, although imports do not nest namespaces upward, they do in some sense nest downward. That is, although an imported module never has direct access to names in a file that imports it, using attribute qualification paths it is possible to descend into arbitrarily nested modules and access their attributes. For example, consider the next three files. First, nest3.py of Example 23-7 defines a single global name and attribute by assignment. Example 23-7. nest3.py

python X = 3 Next, nest2.py of Example 23-8 defines its own X, then imports nest3 and uses name qualification to access the imported module's attribute. Example 23-8. nest2.py

python X = 2 import nest3 print(X, end='') # My global X print(nest3.X) # nest3's X And at the top, nest1.py of Example 23-9 also defines its own X, then imports nest2, and fetches attributes in both the first and second files. Example 23-9. nest1.py

python X = 1 import nest2 print(X, end='') # My glob

al X print(nest2.X, end='') # nest2's X print(nest2.nest3.X) # Nested nest3's X Really, when nest1 imports nest2 here, it sets up a two-level namespace nesting. By using the path of names nest2.nest3.X, it can descend into nest3, which is nested in the imported nest2. The net effect is that nest1 can see the Xs in all three files, and hence has access to all three global scopes:

```
python $ python3 nest1.py 2 3 1 2 3
```

The reverse, however, is not true: nest3 cannot see names in nest2, and nest2 cannot see names in nest1. This example may be easier to grasp if you don't think in terms of namespaces and scopes, but instead focus on the objects involved. Within nest1, nest2 is just a name that refers to an object with attributes, some of which may refer to other objects with attributes (import is an assignment). For paths like nest2.nest3.X, Python simply evaluates from left to right, fetching attributes from objects along the way. Note that nest1 can say import nest2, and then nest2.nest3.X, but it cannot say import nest2.nest3—this syntax invokes something called package (directory) imports, the topic we'll take up in the next chapter after we study reloads here. Package imports also create module namespace nesting, but their import statements are taken to reflect directory trees, not simple file import chains.

中文翻译

最后，虽然导入不会让命名空间"向上"嵌套，但某种意义上它确实会"向下"嵌套。也就是说，虽然被导入的模块永远无法直接访问导入方文件里的名字，但通过属性限定路径，我们可以向下深入任意层级的嵌套模块并访问它们的属性。例如，考虑下面三个文件。首先，示例 23-7 的 `nest3.py` 通过赋值定义一个全局名字和属性。示例 23-7. `nest3.py`

```
python X = 3
```

接着，示例 23-8 的 `nest2.py` 定义自己的 `X`，然后导入 `nest3`，并用名字限定访问被导入模块的属性。示例 23-8. `nest2.py`

```
python X = 2 import nest3 print(X, end='') # 我的全局 X print(nest3.X) # nest3 的 X 最顶层
```

，示例 23-9 的 `nest1.py` 也定义了自己的 `X`，然后导入 `nest2`，并获取前两个文件中的属性。示例 23-9. `nest1.py`

```
python X = 1 import nest2 print(X, end='') # 我的全局 X print(nest2.X, end='') # nest2 的 X print(nest2.nest3.X) # 嵌套的 nest3 的 X
```

事实上，当 `nest1` 在这里导入 `nest2` 时，就建立了一个两层的命名空间嵌套。通过名字路径 `nest2.nest3.X`，它可以向下深入被导入的 `nest2` 中嵌套着的 `nest3`。净效果是：`nest1` 能看到全部三个文件里的 `X`，因而能访问全部三个全局作用域：`python $ python3 nest1.py 2 3 1 2 3` 但反过来则不成立：`nest3` 看不到 `nest2` 里的名字，`nest2` 也看不到 `nest1` 里的名字。如果你不把它们想成命名空间和作用域，而是专注于涉及的对象，这个例子会更容易理解。在 `nest1` 内部，`nest2` 只是一个名字，引用着一个带属性的对象，其中有些属性又可能引用着其他带属性的对象（`import` 就是一次赋值）。对于 `nest2.nest3.X` 这样的路径，Python 只是从左到右求值，沿途从对象

上取属性。注意：nest1 可以说 import nest2，然后写 nest2.nest3.X，但不能写 import nest2.nest3——这个语法调用的是所谓的包（package，即目录）导入，那是我们本章学完 reload 之后、下一章要讲的主题。包导入同样会创建模块命名空间嵌套，但它的 import 语句被理解为反映目录树，而不是简单的文件导入链。

深度理解

- 核心概念：命名空间嵌套是单向向下的：导入方可以"深挖"被导入模块的属性链，被导入方对导入方一无所知。
- 底层实现：把 import 想成一次赋值，一切豁然开朗：import nest2 把模块对象赋给 nest1 作用域里的名字 nest2；而 nest3 恰好是 nest2 的属性之一（因为 nest2.py 顶层执行过 import nest3）。于是 nest2.nest3.X 就是：先取 nest1 的 nest2 → 取其属性 nest3 → 再取其属性 X。整条链是"对象 → 属性"的爬行，不是作用域查找。
- 设计原因：这种"对象嵌套"而非"作用域嵌套"的设计，让深路径访问变得机械而统一——任何带属性的对象都能组成这样的链，无论是模块还是列表的方法。
- 实际问题：现实中你天天在用它：numpy.linalg.eig、os.path.join、urllib.request.urlopen——os 模块里 import 了 path 子模块，于是 os.path 成为 os 的属性。
- 初学者误区：①以为"import 了 nest2 就等于 import 了 nest3"，可以直接用裸名 nest3.X——不行！nest3 只作为属性存在于 nest2 上；②以为 import nest2.nest3 是这种链的简写——不是！那是包（目录）导入语法，语义

完全不同，下章专讲。

代码分析 (Examples 23-7/8/9: 三层嵌套)

```
X = 3
```

```
X = 2
```

```
import nest3
```

```
print(X, end=' ') # X
```

```
print(nest3.X)    # nest3 X
```

```
X = 1
```

```
import nest2
```

```
print(X, end=' ') # X
```

```
print(nest2.X, end=' ') # nest2 X
```

```
print(nest2.nest3.X) # nest3 X
```

- 解释器如何处理：python3 nest1.py 运行：
- 第一行 print 输出 1 (nest1 自己的 X) ；
- import nest2 触发 nest2.py 的执行，它打印 2 3 (nest2 的 X 和 nest3.X)，执行完后 nest2 模块对象及其属性 (含 nest3) 就位；
- 回到 nest1 的 print(nest2.X, end='') 输出 2；
- print(nest2.nest3.X) 沿属性链取出 3。最终输出 2 3 (嵌套文件打印) + 1 2 3 (nest1 打印)。
- 设计思想：用"对象爬行"的眼光看，nest2.nest3.X 与 L.append 没有本质区别——都是沿着对象图取属性。这解释了为什么 Python 不需要单独的"命名空间访问符"：点号通吃。

23.12 重载模块 (Reloading Modules)

英文原文

As we've seen, a module's code is run only once per process by default. To force a module's code to be reloaded and rerun, you need to ask Python to do so explicitly by calling the reload built-in function, introduced briefly in Chapter 3. In this section, we'll explore how to use reloads to make your systems more dynamic. In a nutshell: Imports—via both import and from statements—load and run a module's code only the first time the module is imported in a process. Later imports use the already loaded module object without reloading or rerunning the file's code. This is true even if you resave the module's source code file during the program run. The reload function forces an already loaded module's code to be reloaded and rerun. Assignments in the file's new code change the existing module object in place. So why care about reloading modules? In short, REPL testing and customization. When testing code interactively, it may be easier to reload a module you've changed in another window than it is to restart the REPL. The more grandiose rationale is dynamic customization: the reload function allows parts of a program to be changed without stopping the whole program. With reload, the effects of changes in components can be observed immediately.

ly. Reloading doesn't help in every situation, but where it does, it makes for a much shorter development cycle. For instance, imagine a database program that must connect to a server on startup; because program changes or customizations can be tested immediately after reloads, you need to connect only once while debugging. Long-running servers can update themselves this way, too. Because Python is interpreted (more or less), it already gets rid of the compile/link steps you need to go through to get a C program to run: modules are loaded dynamically when imported by a running program. Reloading offers a further performance advantage by allowing you to also change parts of running programs without stopping. One note here: our use of `reload` in this book is limited to modules written in Python. Compiled extension modules coded in a language such as C can be dynamically loaded by imports at runtime, too, but they are out of scope here (though most users probably prefer to code customizations in Python anyhow!).

中文翻译

正如我们所见，默认情况下模块代码在每个进程中只运行一次。要强制模块代码被重新加载并重新运行，你需要显式请求 Python——调用第 3 章简单介绍过的 `reload` 内置函数。本节我们将探索如何用 `reload` 让系统变得更动态。简单概括：- 导入——无论是 `import` 语句还是 `from` 语句——只在模块于进程中首次被导入时加载并运行其代码。之后的

导入使用已加载的模块对象，不会重载或重跑文件代码。即使程序运行期间你重新保存了模块的源代码文件，也是如此。 - reload 函数强制一个已加载模块的代码被重新加载并重跑。文件新代码中的赋值会就地修改现有的模块对象。那为什么要在意重载模块？简而言之：REPL 测试和定制。交互式测试代码时，在另一个窗口修改了模块后，重载它比重启 REPL 要省事得多。更宏大的理由是动态定制（dynamic customization）：reload 函数允许在不停止整个程序的情况下修改程序的一部分。有了 reload，组件修改的效果可以立即被观察到。重载并非在所有情况下都有用，但在有用的地方，它大大缩短了开发周期。例如，设想一个启动时必须连接服务器的数据库程序；因为修改或定制可以在 reload 之后立即测试，调试时你只需要连接一次服务器。长期运行的服务器也可以这样自我更新。因为 Python（或多或少）是解释型语言，它已经省去了让 C 程序运行所需的编译/链接步骤：模块在运行程序 import 它们时被动态加载。重载进一步带来性能上的好处——它还允许你在不停止程序的情况下修改运行中程序的一部分。这里有一点要说明：本书中 reload 的使用仅限于用 Python 写的模块。用 C 等语言编写的已编译扩展模块同样可以在运行时被 import 动态加载，但不在本书讨论范围内（何况大多数用户大概也更愿意用 Python 来写定制逻辑！）。

深度理解

·核心概念：import 是"一次性缓存"，reload 是"强制重跑"。reload 解决了"导入只发生一次"带来的开发痛点。

·底层实现：reload 的实现逻辑是：重新按 sys.path 查找模块文件 → 重新编译 → 在同一个模块对象里重跑顶层代

码（重用模块原有的 dict，而不是新建对象）。这就是“就地修改”（in place）的底层含义。

·设计原因：为什么不做成“自动检测文件变化就重载”？因为自动重载有状态一致性问题（旧引用、正在进行的计算）；Python 把选择权交给程序员——显式 reload，其他一切都保持不变。

·实际问题：开发循环从“改代码 → 重启进程 → 重新连数据库”缩短为“改代码 → reload → 立即验证”；长驻服务（服务器、守护进程）可以借此实现不停机更新。

·初学者误区：①以为改了源码再 import 就会生效——不会，必须 reload；②以为 reload 后所有引用都自动更新——只有模块对象本身更新，from 复制出来的旧引用纹丝不动。

23.12.1 reload 基础 (reload Basics)

英文原文

Unlike import and from: - reload is a function in Python, not a statement. - reload is passed an existing module object, not a string name. - reload lives in a standard-library module and must be imported itself. Because reload expects an object, a module must have been previously imported successfully before you can reload it (and if the import was unsuccessful due to a syntax or other error, you may need to repeat it before you can reload the module). Furthermore, the syntax of import statements and reload calls differs: as a

function, reloads require parentheses, but import statements do not. Abstractly, reloading looks like this:

```
python import module # Initial module import ... use module.attributes...
```

```
... change the module file
```

```
... from importlib import reload # Get reload itself reload(module) # Get updated module
```

```
... use module.attributes...
```

The typical usage pattern is that you import a module, then change its source code in a text editor, and then reload it. This can occur when working interactively, but also in larger programs that reload periodically. When you call `reload`, Python rereads the module's code and reruns its top-level statements. Perhaps the most important thing to know about `reload` is that it changes a module object in place; it does not delete and re-create the module object. Because of that, every reference to an entire module object anywhere in your program is automatically affected by a reload. Here are the details:

- `reload` runs a module file's new code in the module's current namespace. Rerunning a module file's code overwrites its existing namespace, rather than deleting and re-creating it.
- Top-level assignments in the file replace names with new values. For instance, rerunning a `def` statement replaces the prior version of the function in the module's namespace by reassigning the function name.
- Reloads impact

ct all clients that use import to fetch modules. Because clients that use import qualify to fetch attributes, they'll find new values in the module object after a reload.- Reloads impact future from clients only. Clients that used from to fetch attributes in the past won't be affected by a reload; they'll still have references to the old objects fetched before the reload. - Reloads apply to a single module only. You must run them on each module you wish to update unless you use code or tools that apply reloads transitively.

中文翻译

与 import 和 from 不同：- reload 是 Python 中的函数，不是语句。- reload 接收的是一个已存在的模块对象，而不是字符串名字。- reload 住在一个标准库模块里，必须先导入它本身。因为 reload 期望的是一个对象，所以你必须先成功导入过某个模块，才能重载它（如果导入因语法错误或其他错误而失败，你可能需要先修复并重新导入，然后才能重载）。此外，import 语句与 reload 调用的语法也不同：作为函数，reload 需要括号，而 import 语句不需要。抽象地看，重载的流程是这样的：python import module # 最初的模块导入...使用 module.attributes... ...修改模块文件... from importlib import reload # 取得 reload 本身reload(module) # 取得更新后的模块...使用 module.attributes... 典型的用法是：先导入模块，然后在文本编辑器里修改它的源代码，再重载它。这可以发生在交互式工作中，也可以发生在定期重载的大型程序中。调用 reload 时，Python 会重新读取模块代码并重跑其顶层语句。关于 re

load 最重要的一点也许是：它就地修改模块对象；它不会删除并重新创建模块对象。正因为如此，程序中任何地方对整个模块对象的引用都会自动受到一次 reload 的影响。细节如下：

- reload 在模块当前命名空间中运行文件的新代码。重跑模块文件代码会覆盖其现有命名空间，而不是删除并重建它。
- 文件中的顶层赋值用新值替换名字。例如，重跑一条 def 语句会通过重新赋值函数名，替换掉模块命名空间中旧版本的函数。
- reload 影响所有用 import 获取模块的使用者。因为用 import 的使用者是通过限定访问来取属性的，reload 后他们会在模块对象中找到新值。
- reload 只影响未来的 from 使用者。过去用 from 取过属性的使用者不受 reload 影响；他们仍然持有 reload 前取到的旧对象的引用。
- reload 只作用于单个模块。除非你使用代码或工具对相关模块进行传递式重载，否则每个想更新的模块你都得各自调用一次。

深度理解

·核心概念：reload 是"函数"而非"语句"，且是就地更新——复用同一个模块对象。这五个细节是 reload 的全部行为规范。

·底层实现：reload 复用模块对象的命名空间字典：新代码的顶层赋值直接覆盖旧键；旧模块对象在 sys.modules 中的位置不变，任何指向该对象的名字（import 产生的引用）都自动看到新值。而 from 复制出去的名字是"值引用"（指向旧函数对象/旧值），不在模块对象里，自然不受影响。

·设计原因：为什么不删除重建？因为那会破坏所有现存引

用（其他模块持有的模块对象、全局状态、正在运行的计算），就地更新则把影响面控制在"通过模块对象访问的人"。

·实际问题：这就是开发服务器热重载（如 Flask 的 debug 模式、Jupyter 的 autoreload 扩展）的原理原型；也是"reload 与 from 不兼容"问题的根源——from 拿到的是旧对象的拷贝，reload 对它无效。

·初学者误区：①以为 reload 要传字符串名——错，要传模块对象（可用 sys.modules['name'] 取）；②以为 reload 会连带重载该模块 import 的其他模块——不会，只更新当前一个；③以为 from 来的名字也会更新——不会，只影响未来从模块对象取属性的操作。

代码分析（reload 抽象流程）

```
import module                                #
...
module.attributes...
...

...
from importlib import reload # reload
reload(module)                #
...
module.attributes...
```

·解释器如何处理：from importlib import reload 是标准库导入——reload 在 Python 3.4 起定居于 importlib 模块（旧版本里是内置函数或 imp 模块的属性）。reload(module) 执行时，解释器按模块对象的源文件路径重新读取、重新编译、重新执行顶层语句，所有赋值覆盖原命

名空间，然后返回更新后的模块对象。

·设计思想：注意"import 语句不需要括号、reload 函数需要括号"这个区别——这是语言设计上"语句 vs 表达式"的经典分工：语句负责声明式行为，函数负责可计算的行为。

23.12.2 reload 示例 (reload Example)

英文原文

To demonstrate, here's a more concrete example of reload in action. In the following, we'll change and reload a module file without stopping the interactive Python session. Reloads are useful in other scenarios, too, but we'll keep things simple for illustration here. First, in the text editor of your choice, write a module file named `changer.py` with the contents given in Example 23-10. Example 23-10. `changer.py` (start)

```
python message = 'First version'
def printer(): print(message)
This module creates and exports two names—one bound to a string, and another to a function. Now, start the Python interpreter (i.e., your local REPL), import the module, and call the function it exports. The function will print the value of the global message variable as you probably expect:
```

```
python $ python3 >>> import changer >>> changer.printer()
First version
```

Keeping the interpreter active, now edit and save the module file in another window. Change the global message variable, as well as the printer function body:

```
python message = 'After editing'
def printer(): print('reloaded:', message)

```

Then, return to the Python window and reload the module to fetch the new code. Notice in the following interaction that importing the module again has no effect; we get the original message, even though the file's been changed. We have to call reload in order to get the new version:

```
python >>> import changer >>> changer.printer() #
No effect: uses loaded module
First version

>>> from importlib import reload >>> reload(changer) # Forces new code to load/run
<module'changer' from '/.../LP6E/Chapter23/changer.py'>

>>> changer.printer() # Runs the new version now reloaded: After editing

```

Notice that reload actually returns the module object for us—its result is usually ignored, but because expression results are printed at the interactive prompt, Python shows a default `<module ...>` representation. You can also reload a module by string name using the `sys.modules` dictionary demoed in the prior chapter if you don't have a variable assigned to it by import:

`python >>> import sys >>> reload(sys.modules['changer'])` In this case, it's probably easier to run `import changer` to get a handle on the module, but the `sys.modules` scheme may be useful in programs that need to reload modules more generically. It's also possible to delete modules from `sys.modules` to force a reload, but this is generally discouraged for reasons we'll skip here; use `reload`.

中文翻译

为了演示，这里给一个更具体的 `reload` 实战例子。下面我们会在不停止交互式 Python 会话的情况下修改并重载一个模块文件。`reload` 在其他场景下也有用，但为了演示这里我们尽量简单。首先，在你喜欢的文本编辑器里，按示例 23-10 的内容写一个名为 `changer.py` 的模块文件。示例 23-10. `changer.py` (初始版)

```
python message = 'First version'
def printer(): print(message)
```

这个模块创建并导出两个名字——一个绑定到字符串，另一个绑定到函数。现在启动 Python 解释器（也就是你的本地 REPL），导入这个模块，并调用它导出的函数。函数会打印全局变量 `message` 的值，正如你预料的那样：`python $ python3 >>> import changer >>> changer.printer()` `First version` 保持解释器运行，现在在另一个窗口编辑并保存模块文件。修改全局变量 `message`，也修改 `printer` 的函数体：

```
python message = 'After editing'
def printer(): print('reloaded:', message)
```

然后回到 Python 窗口，重载模块以取得新代码。注意下面这次交互：再次 `import` 模块没有效果；即使文件已经改了，我们得到的还是原来的 `message`。必须调

用 reload 才能拿到新版本: `python >>> import changer >>> changer.printer()` # 无效: 用的是已加载的模块
First version `>>> from importlib import reload >>> reload(changer)` # 强制加载/运行新代码
<module'changer' from'../LP6E/Chapter23/changer.py'> `>>> changer.printer()` # 现在运行的是新版本
reloaded: After editing
注意 reload 实际上会把模块对象返回给我们——它的结果通常被忽略, 但因为交互式提示符会打印表达式结果, Python 就显示了默认的 <module ...> 表示。如果你没有通过 import 获得模块变量, 也可以用上一章演示过的 `sys.modules` 字典按字符串名字重载模块: `python >>> import sys >>> reload(sys.modules['changer'])` 这种情况下, 运行 `import changer` 拿个模块句柄可能更简单, 但 `sys.modules` 方案对需要更通用地重载模块的程序可能有用。也可以从 `sys.modules` 中删除模块来强制重载, 但这通常不被鼓励, 原因这里先跳过; 请用 reload。

深度理解

- 核心概念: 完整演示"改文件 → 重 import 无效 → reload 生效"的完整闭环。这是全章实践性最强的部分。
- 底层实现: 第二次 `import changer` 时 `sys.modules['changer']` 命中, 直接返回旧模块对象——所以打印的还是 First version; `reload(changer)` 则重新读文件、重编译、重执行, `message` 和 `printer` 两个绑定被覆盖为新值, 于是后续调用打印 reloaded: After editing。
- 设计原因: reload 返回模块对象是为了配合交互环境 (REPL 会打印表达式结果), 也方便链式使用; "从 `sys.mo`

dules 删除再 import"之所以被劝阻，是因为残留引用可能指向已删除的模块，导致诡异行为。

·实际问题：多窗口开发（一个编辑器、一个 REPL）的标准工作流；sys.modules['name'] 的按名重载方式适合写通用重载工具（遍历 sys.modules 批量重载）。

·初学者误区：①忘了 reload 需要先 import；②在 REPL 里直接 reload('changer') 传字符串——TypeError，要传模块对象；③以为 reload 会更新旧函数对象的所有引用——旧 printer 函数对象的引用仍指向旧函数。

代码分析 (Example 23-10: 完整 reload 会话)

```
message = 'First version'
def printer():
    print(message)
```

```
$ python3
>>> import changer
>>> changer.printer()
First version
```

```
>>> import changer
>>> changer.printer()           #
First version
>>> from importlib import reload
>>> reload(changer)             # /
<module 'changer' from '/.../LP6E/Chapter23/changer.py'>
>>> changer.printer()           #
reloaded: After editing
```

·解释器如何处理：整个会话中 changer 一直指向同一个模块对象（reload 就地更新）。变化只发生在模块对象的命名空间里：message 键的值从'First version' 换成'Aft

er editing', printer 键的函数对象被新函数替换。所以"通过模块对象访问"的一切 (changer.printer()) 看到新版本。

·设计思想: reload 是"热更新" (hot swap) 的最小实现: 不变对象、只换内容。生产级热重载 (如 gunicorn 的 reload、Django dev server) 在此原理上加了文件监听、依赖追踪和进程管理。

23.12.3 reload 杂项 (reload Odds and Ends)

英文原文

Finally, four brief module-reload notes in closing:- If you use reload, you'll probably want to pair it with import instead of from, as names fetched with the latter are not updated by reload operations—leaving your names in a state that's strange enough to warrant postponing elaboration until this part's "gotchas" at the end of Chapter 25.- By itself, reload updates only a single module, but it's straightforward to code a function that applies it transitively to related modules—an extension we'll save for a case study near the end of Chapter 25.- Some development tools (e.g. Jupyter notebooks) have REPLs that offer an auto-reload mode that may obviate manual reload calls. This works only in specific tools, though, and doesn't address customization roles.- And last, reload has had a long and checkered past—morphing from built-in function, to imp module attribute in this book's prior edition, to it

s current importlib host. While this may be a symptom of Python's constant morph, which is apt to relocate reload again, it must be asked: does this function have trouble working with others?!

中文翻译

最后，收尾时给出四条关于模块重载的简短说明：- 如果你使用 reload，最好搭配 import 而不是 from，因为用后者取来的名字不会被 reload 操作更新——这会让你的名字处于一种够奇怪的状态，值得推迟到本部分末尾第 25 章的"陷阱"里再展开讲。- reload 本身只更新单个模块，但写一个对相关模块传递式应用它的函数是件直截了当的事——这个扩展我们留到第 25 章末尾的案例研究里。- 一些开发工具（例如 Jupyter notebook）的 REPL 提供自动重载（auto-reload）模式，可能让你无需手动调用 reload。不过这只在特定工具中有效，也解决不了定制化的需求。- 最后，reload 有着漫长而曲折的历史——从内置函数，变成本书前一版里 imp 模块的属性，再到现在寄居在 importlib 里。虽然这可能是 Python 不断"变身"的症状之一，而且 reload 将来可能再次搬家，但还是要问一句：这个函数是不是和人相处不来？！

深度理解

·核心概念：四条使用纪律——①reload 配 import 不配 from；②只重载单模块（传递式要自己写）；③工具可自动重载；④reload 的历史包袱。

·底层实现：为什么 from 与 reload 犯冲？from 复制的是"重载前的引用"，reload 只改模块对象的绑定——你手

里的旧引用自然过期。这是引用语义的直接推论。

·设计原因：reload 的历史迁移（内置 → imp → importlib）反映了 Python 导入系统从“内置魔法”走向“可插拔标准库模块”（PEP 451 的 importlib 重构）的演进。

·实际问题：现代项目里，Jupyter 的 %autoreload、IPython 的 reload magic、pytest 的 --pyargs 场景、Flask /uvicorn 的 --reload 都与此直接相关。

·初学者误区：以为 reload 能“覆盖一切更新需求”——它对已存在模块对象内容有效，但对模块依赖关系、from 引用、类实例（旧实例仍用旧类）都无能为力；生产环境热更新请优先考虑进程重启或成熟框架。

23.13 章节总结 (Chapter Summary)

英文原文

This chapter drilled down into the essentials of module coding tools—the import and from statements, and the reload call. It showed how the from statement simply adds an extra step that copies names out of a file after it has been imported, and how reload forces a file to be imported again without stopping and restarting Python. This chapter also detailed what happens when imports are nested, explored the way files become module namespaces, and covered some potential pitfalls of the from statement. Although you've already learned enough to handle module files in mo

st programs, the next chapter extends the coverage of the import model by presenting package imports—a way for import statements to specify part of the directory path leading to the desired module. As you'll find, package imports give us a hierarchy that is useful in larger systems and allow us to break conflicts between same-named modules. Before we move on, though, here's a quick quiz on the concepts presented here.

中文翻译

本章深入讲解了模块编码工具的精髓——import 和 from 语句，以及 reload 调用。它展示了 from 语句如何只是多加一步、在文件被导入后把名字复制出来，也展示了 reload 如何在不停止和重启 Python 的情况下强制文件被再次导入。本章还详述了导入嵌套时发生什么，探索了文件变成模块命名空间的方式，并讨论了 from 语句的一些潜在陷阱。虽然你已经掌握了处理大多数程序中模块文件所需的全部知识，但下一章将介绍包导入（package imports）来扩展导入模型——它让 import 语句可以指定通往目标模块的部分目录路径。你会看到，包导入提供了对大型系统很有用的层级结构，并允许我们打破同名模块之间的冲突。不过在继续之前，这里有一个关于本章概念的小测验。

深度理解

·承上启下：本章是模块知识的“地面部队”——单文件层面的导入、属性、重载；第 24 章是“空中支援”——目录层面（包）的导入体系，解决同名模块冲突和大型系统组织

知识串线：把本章三条主线连起来看：from = import + 复制名字（等价性）、模块 = 文件 + 命名空间字典（底层实现）、reload = 就地重跑（动态更新）。这三条线覆盖了模块使用的 90% 场景。

23.14 自测：Quiz（随章测验）

测验题目（中英对照）

1. How do you make a module?如何创建一个模块？
2. How is the from statement related to the import statement? from 语句与 import 语句是什么关系？
3. How is the reload function related to imports? reload 函数与导入是什么关系？
4. When must you use import instead of from?什么时候你必须用 import 而不用 from？
5. Name three potential pitfalls of the from statement.说出 from 语句的三个潜在陷阱。

参考答案（Answers）

1. To create a module, you simply write a text file containing Python statements; every source code file is automatically a module, and there is no syntax for declaring one. Import operations load module files into module objects in memory. You can also make a module by writing code in an external language like C or Java, but such extension modules are beyond the scope of this book (and most of its readers).要创建一个模

块，你只需写一个包含 Python 语句的文本文件；每个源代码文件都自动是一个模块，没有任何声明模块的语法。导入操作把模块文件加载为内存中的模块对象。你也可以用 C 或 Java 等外部语言编写代码来制作模块，但这类扩展模块超出了本书（以及大多数读者）的范围。

2. The `from` statement imports an entire module, like the `import` statement, but as an extra step, it also copies one or more variables from the imported module into the scope where the `from` appears. This enables you to use the imported names directly (`name`) instead of having to go through the module (`module.name`). `from` 语句和 `import` 语句一样会导入整个模块，但它额外多走一步：把被导入模块中的一个或多个变量复制到 `from` 语句所在的作用域。这让你可以直接使用导入的名字 (`name`)，而不必经过模块 (`module.name`)。

3. By default, a module is imported only once per process. The `reload` function forces a module to be imported again. It is mostly used to pick up new versions of a module's source code during development, and in dynamic customization scenarios where users change part of a system without restarting it. 默认情况下，模块在每个进程中只被导入一次。`reload` 函数强制模块被再次导入。它主要用于开发过程中加载模块源代码的新版本，以及在用户不改动整个系统的前提下修改其中一部分的动态定制场景。

4. You must use `import` instead of `from` only when you need to access the same name in two different modules. To make the two names unique, qualify with the names of their enclosing modules obtained with `import`. The as

extension can render from usable in this context as well, by renaming imports uniquely.只有当你需要在两个不同模块中访问同一个名字时，才必须用 import 而不是 from。为了让两个名字唯一，要用 import 取得所在模块名后再限定访问。as 扩展也能通过唯一重命名让 from 在这种场景下可用。5. The from statement can obscure the meaning of a variable (which module it is defined in), can have problems with the reload call (names may reference prior versions of objects), and can corrupt namespaces (it might silently overwrite names you are using in your scope). The from form is worse in most regards—it can corrupt namespaces arbitrarily and obscure the meaning of variables, so it is probably best used sparingly. from 语句可能让变量的含义变得模糊（看不出它定义在哪个模块里），可能与 reload 调用产生问题（名字可能引用旧版本的对象），还可能污染命名空间（可能静默覆盖你作用域中正在使用的名字）。from 形式在大多数方面更糟——它可以随意破坏命名空间、模糊变量的含义，所以最好克制使用。

本章技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

·模块即项目骨架：任何 Python 项目的根目录都是一堆模块文件（config.py、db.py、utils.py），import 是项目内部复用的唯一通道。理解“顶层赋值即导出”就能设计出

干净的公共 API 面。

- `if name == 'main':` 双用途文件：这是本章 `name` 两种状态（被导入时 = 模块名；被直接运行时 = `'main'`）的直接应用——同一文件既能被 `import` 复用其函数，也能当脚本 `python file.py` 运行其测试/入口代码。现代写法甚至引入 `if name == 'main': main()` 惯例。

- `sys.path` 定制：临时目录、`PYTHONPATH` 环境变量、`sys.path.insert(0, ...)` 都是真实项目里让“别的目录的模块也能被 `import`”的常用手段。但更规范的做法是用包（第 24 章）或把项目做成可安装包（`pyproject.toml` / `site-packages`）。

- 插件系统：动态列出模块 `dict(dir(module))`、用 `importlib` 按字符串名字导入（`importlib.importmodule('pkg.mod')`）并遍历注册——这是插件框架（`pytest` 插件、`Flask` 扩展）的底层套路，本章的 `dict` 部分就是起点。

- 开发热重载：调试服务器、`notebook`（`%autoreload`）、`Flask`/`uvicorn` 的 `--reload` 都是 `reload` 思想的工程化；生产环境的热更新仍优先考虑无状态设计与滚动重启。

- 命名规范落地：团队规范里“禁止 `from x import`”（`lint` 默认报错）、“`import` 一律放顶部”（`PEP 8 E402` 之外还有 `import` 排序工具 `isort/ruff`）都是本章陷阱章节的实践化。

2. 与其他语言的区别 (Java/C++)

维度	Python (import/from)	Java (import)	C/C++ (#include)
加载时机	运行时（执行到才加载）	编译期 + 运行期类加载（JVM	编译期（预处理拼接/链接期）
导入单位	模块对象（可执行语句）	类/包	头文件/翻译单元
名字隔离	强限定（module.attr）	包名限定 + 可选静态导入	宏/命名空间（可选）
缓存	sys.modules 每进程一次	JVM 类加载器缓存	无（每编译单元展开）
动态重载	importlib.reload 就地更新	无（自定义 ClassLoader 很复	无（需重新编译链接）
条件/按需导入	语句可放任何位置（if/try/def	只能文件顶部	只能在预处理阶段
命名空间分区	模块 = 命名空间，天然分区	package 强分区（访问控制）	命名空间可选，易冲突

核心洞察：Python 把"导入"做成运行时可执行语句，换来的是动态加载、条件导入、热重载的自由；Java 把 import 限定为"编译期名字解析"，换来静态可分析性；C 的 #include 则是文本级拼接，最原始也最易冲突。三者的根本差异在于：名字解析发生在哪个阶段。

3. Python 发展历史背景

- 模块系统是 Python 的"原初设计"：1991 年的 Python 0.9 就已包含模块、异常、函数三大核心——模块从一开始就是一等公民，而非后来的补丁。
- import 机制演进：早期 import 逻辑写死在解释器里；PEP 302（2003）引入 finder/loader 可插拔架构（zip 导入、钩子成为可能）；PEP 451（2013，Python 3.4）以 importlib 全面重构导入系统，reload 也随之从内置函数迁入 importlib 模块（此前它是 imp.reload，Pytho

n 3.12 中 imp 已弃用)。

- 字节码缓存：模块首次编译成字节码后会写入 `pycache/module.cpython-312.pyc`，后续导入直接加载 `.pyc`——这是“导入昂贵、所以要缓存”的物理基础，也是“改 `.py` 后时间戳校验失效会重新编译”的机制。

- `__name__ == 'main'` 惯用法：源自 Python 早期“脚本与模块统一”的设计哲学——同一个文件既可以运行也可以被导入，判断依据就是这个特殊变量，它从 1.x 时代沿用至今。

4. 高级开发者需要掌握的相关知识

- `importlib` 全链路：`importlib.importmodule`（字符串动态导入）、`importlib.util.findspec`（探测模块是否存在）、`importlib.reload`——动态导入是插件系统与延迟加载（lazy import）的基础。

- `sys.modules` 与 `sys.path` 的运维视角：`sys.modules` 是调试利器（看谁加载了、何时加载的）；`sys.path` 的注入（`site-packages`、`.pth` 文件）决定了环境装配。

- 字节码层：用 `dis.dis('import x')` 观察 `import` 的字节码，理解它“赋值”的本质；`pycache` 与 `.pyc` 校验是构建与部署（`Docker` 镜像分层、`frozen modules`）的常识。

- PEP 562（模块级 `getattr`）：Python 3.7 起模块可以定义 `getattr`，实现“延迟属性/移除属性警告”——模块从“静态字典”进化为“可编程对象”。

- 类型标注与静态检查：`import` 的模块类型注解（`from fu`

ture import annotations)、mypy 对 init.pyi 存根文件的处理——动态导入如何与静态分析共存是大型项目的必修课。

本章学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5)。模块是 Python 组织的唯一骨架，本章（连同 22、24 章）是"多文件程序"的全部基础；name、reload、from 语义在面试与实战中出现频率极高。
- 应该掌握到什么程度：
- 必会：用文本解释 import vs from vs from 三者的差别与适用场景；说出"模块只加载一次"及其机制 (sys.modules)；能讲清 from 的"复制引用"语义 (改名字 vs 改对象)；会用 dir()/dict 查看模块内容；会用 importlib.reload 并知道它不更新 from 来的名字。
- 应会：if name == 'main': 惯用法；as 别名；sys.path 的组成与修改方式；模块命名规范（小写下划线、避开标准库名）。
- 了解：扩展模块 (C 扩展) 的存在与用途；all 与 X 数据隐藏 (第 25 章详解)；from 与 reload 冲突的"gotchas"细节 (第 25 章)。
- 后续学习内容：
- 立即：第 24 章"包导入"——目录级别的模块组织，import pkg.mod 与 init.py；

·近程：第 25 章"模块高级话题"——all、as、重载陷阱案例、相对导入；

·进阶（学完本书后）：importlib 源码、PEP 302/451 文档、CPython Modules/import.c 与 Lib/importlib 的对照阅读。

·实践建议：开两个窗口——一个编辑器、一个 REPL，亲手复现本章全部示例：创建 module1.py、share.py、spaces.py、changer.py，依次验证"import 限定访问 / from 复制语义 / dict 内省 / reload 热更新"四个实验；再用 import sys; print(sys.path) 与 print(sys.modules) 亲眼看看"缓存表"长什么样。

本章完。下一章：Package Imports（包导入）——目录级别的模块组织。